

GITHUB OPEN-SOURCE RESEARCH REPORT

The Ultimate LLM Harness Arsenal

A curated, verified sweep of the best open-source projects across every capability a world-class agent harness needs — memory, token efficiency, web scraping, file search, browser use, TTFT/inference, orchestration, tools/MCP, RAG, prompt engineering, evals, planning, and reference harnesses to study.

14

categories

100+

verified projects

13

parallel research agents

Compiled 2026-09-17 · every repo & star count verified against live GitHub

Contents

★ User-Flagged References (ECC · TencentDB Agent Memory · awesomeclaude.ai)

1 Existing Agent Harnesses & Coding CLIs

2 LLM Memory & Knowledge

3 Token Efficiency & Context Compression

4 Web Scraping & Crawling

5 Mass File Search & Code Navigation

6 Browser Automation & Computer Use

7 Inference Speed, TTFT & Local Serving

8 Agent Orchestration & Multi-Agent Frameworks

9 Tool Use, MCP & Sandboxed Execution

10 RAG, Embeddings & Retrieval

11 Prompt Engineering & Optimization

12 Evaluation, Observability & Tracing

13 Planning, Reasoning & Self-Improvement

★ User-Flagged References

Three references you flagged as exemplary — treated as priority study material since they represent the exact "harness as a product" philosophy you're targeting.

ECC — Everything Claude Code (affaan-m/ECC)

- **Repo:** github.com/affaan-m/ECC · index: awesomeclaude.ai/everything-claude-code
- **Stars:** ~261k (forks ~39k) — one of the most-starred agent-harness projects on GitHub
- **Language:** Markdown/TypeScript (plugin-based, model-agnostic)
- **What it does:** A complete "agent harness performance optimization system" shipping **619 components**: 67 agents, 281 skills, 94 commands, 121 rules, 21 hooks, and 35 MCP servers — installable as a plugin into Claude Code, Codex, OpenCode, Cursor and more.
- **Why it's the best:** This is the closest existing thing to your goal — a harness layer that makes *any* plugged-in model dramatically better. Standout ideas worth stealing: **hooks** that fire on lifecycle events (PreToolUse/PostToolUse/Stop) to enforce rules automatically; **instincts/memory files** that persist learned preferences; specialized **build-error-resolver** agents per language; a **GAN-style generator/evaluator loop** for self-improving output quality; a **harness-optimizer** agent that tunes the harness itself; and **silent-failure-hunter/comment-analyzer** review agents.
- **Harness integration notes:** Study the directory layout (`agents/` , `skills/` , `commands/` , `rules/` , `hooks/`) as a component taxonomy for your own harness. The hook system and the skill-activation model are the two highest-value patterns to replicate — they convert passive prompt text into enforced behavior.

TencentDB Agent Memory

- **Repo:** github.com/TencentCloud/TencentDB-Agent-Memory
- **Stars:** ~26.9k
- **Language:** Go / Python (server + SDK + adapters)
- **What it does:** A **team-level memory hub** for AI agents — turns conversations, docs, and code into four reusable, governed memory assets: **Chat Memory**, **Skill** (reusable procedures), **LLM-Wiki** (distilled knowledge), and **Code-Graph** (structural code memory). Ships adapters (incl. OpenCode) so multiple agents/frameworks share one memory layer.
- **Why it's the best:** The four-asset model is the most complete taxonomy of what "agent memory" should actually be — most tools stop at chat history; this adds skills, wiki, and code-graph as first-class, shareable assets with governance. The multi-agent *sharing* angle (memory as team infrastructure, not per-session state) is where harnesses are heading.
- **Harness integration notes:** Adopt its asset taxonomy for your memory layer: separate stores for episodic chat, procedural skills, distilled wiki knowledge, and a code graph — with per-asset TTL/governance. Its `MemoryProxy` + adapter pattern is the clean way to let multiple agent frontends share one memory backend.

awesomeclaude.ai — Everything Claude Code Index

- **Site:** awesomeclaude.ai/everything-claude-code
- **What it does:** A searchable index of all 619 ECC components (agents/skills/commands/rules/hooks/MCP servers) with one-line descriptions and direct source links.
- **Why it's useful:** The fastest way to mine ECC for ideas — browse by component type, read the one-liners, and cherry-pick the agents/skills that map to your harness features (e.g. `agent-evaluator` for self-grading, `conversation-analyzer` for turning transcripts into new hooks, `code-explorer` for repo onboarding).
- **Harness integration notes:** Use it as a menu when speccing your own component library; the one-line "Use PROACTIVELY when..." trigger descriptions are a proven pattern for skill auto-activation.

1 Existing Agent Harnesses & Coding CLIs (Reference Designs)

The fastest way to build a world-class harness is to study the ones that already won. Each project below is a complete, battle-tested agent loop with a distinct design philosophy: edit-format engineering (Aider), sandboxed event-sourced autonomy (OpenHands), IDE-native human-in-the-loop (Cline/Roo), terminal-first UX (opencode/Crush), MCP-native extensibility (Goose), and minimal academic agent-computer interfaces (SWE-agent). For each: what to copy, and what to avoid.

opencode

- **Repo:** <https://github.com/anomalyco/opencode> (formerly sst/opencode)
- **Stars:** ~208k
- **Language:** TypeScript (Bun), with a Go TUI lineage
- **What it does:** The most popular open-source terminal coding agent. Client/server architecture: an `opencode` server exposes the agent over HTTP, and the TUI, desktop app, VS Code extension, and `opencode run` headless mode are all just clients.
- **Why it's the best:** The server/client split is the key design — any frontend (TUI, web, IDE, CI script) drives the same agent session, and sessions are shareable/multi-client. Very fast iteration, provider-agnostic via Models.dev config, built-in LSP integration so the agent sees the same diagnostics the editor does, plan/build agent modes, and first-class subagents and MCP support.
- **Harness integration notes:** Copy: headless server + thin clients, LSP-as-a-tool for diagnostics, shareable session URLs. Avoid: the project has churned through rewrites (Go TUI → Bun) and org moves; config schema stability is weak. Its tool surface is large — study which tools actually earn their tokens.

Aider

- **Repo:** <https://github.com/Aider-AI/aider>
- **Stars:** ~49k
- **Language:** Python
- **What it does:** The original terminal pair-programmer. Chat-driven editing of an explicit file set, with automatic git commits, lint/test fix loops, and a tree-sitter "repo map" that gives the model a ranked skeleton of the whole codebase.
- **Why it's the best:** Two ideas here are canonical. (1) **Edit formats:** `aider/coders/` contains ~15 interchangeable "coder" strategies (whole-file, unified diff, search/replace edit-blocks, editor-diff, architect+editor two-model split) selected per-model based on the aider polyglot benchmark — the definitive public dataset on which edit representation each LLM applies most reliably. (2) **Repo map:** tree-sitter tag extraction + PageRank over the symbol reference graph yields a compact, always-relevant codebase context.
- **Harness integration notes:** Copy: pluggable edit-format strategy chosen per model, repo-map ranking, auto-commit-per-change with sane messages, lint/test feedback loop. Avoid: the user-

managed `/add` file-set model — a modern harness should retrieve context autonomously; also the single-threaded chat loop has no real subagent/tool abstraction.

Cline

- **Repo:** <https://github.com/cline/cline>
- **Stars:** ~69k
- **Language:** TypeScript
- **What it does:** Autonomous coding agent that started as a VS Code extension and is now a shared agent core ("Cline Core") driving a VS Code extension, JetBrains plugin, CLI (interactive + headless), desktop app, and a public `@cline/sdk`.
- **Why it's the best:** The reference implementation of **human-in-the-loop autonomy**: every edit is a reviewable diff, every terminal command needs approval (with granular auto-approve rules), and checkpoints snapshot workspace state so any agent step is one-click revertible. Plan/Act mode separation, `.clinerules` project rules, MCP marketplace, and streaming terminal output that the agent reacts to mid-execution (catches compile errors/test failures as they appear).
- **Harness integration notes:** Copy: checkpoint/restore as a first-class primitive, per-action approval policy engine, Plan vs Act system-prompt split, one agent core behind many frontends. Avoid: the VS Code webview message-passing architecture is heavy; the XML-ish tool-call prompt format predates native function calling and shouldn't be copied verbatim.

OpenHands (ex-OpenDevin)

- **Repo:** <https://github.com/All-Hands-AI/OpenHands>
- **Stars:** ~88k
- **Language:** TypeScript frontend; Python agent SDK (software-agent-sdk)
- **What it does:** Full autonomous software-development platform: agents run in sandboxed Docker runtimes with a bash shell, file editor, and browser, coordinated through an event-sourced conversation stream. Now ships "Agent Canvas," a control center that runs OpenHands or any ACP-compatible agent (Claude Code, Codex, Gemini) across local/Docker/VM/cloud backends, plus scheduled/webhook automations.
- **Why it's the best:** The most rigorous **agent runtime architecture** in the open: an append-only Event Stream (actions/observations) is the single source of truth — the agent loop, UI, and evaluation all consume the same events, giving replay, resume, and multi-client views for free. Docker sandboxing per session is the security model everyone else approximates. Also the strongest eval culture (SWE-bench Verified top-tier results).
- **Harness integration notes:** Copy: event-sourced action/observation log as the core abstraction, sandboxed runtime with a small well-designed tool set (bash, str_replace editor, browse), microagent (scoped instruction) system. Avoid: the full stack is heavy (multi-repo, Docker-in-Docker, enterprise features); don't import the platform complexity, just the event model and runtime isolation.

Goose

- **Repo:** <https://github.com/aaif-goose/goose> (originally block/goose)
- **Stars:** ~54k
- **Language:** Rust (core) + TypeScript (desktop UI)
- **What it does:** Block's open-source extensible AI agent — CLI and desktop app — built from day one around MCP: every capability (developer tools, computer controller, memory, JetBrains) is an extension, and users add MCP servers as first-class plugins.
- **Why it's the best:** The cleanest **MCP-native architecture**: the agent core is deliberately small and treats its own built-in tools the same as third-party MCP extensions, with per-extension enablement, tool allowlists, and "recipes" (shareable YAML task definitions bundling prompt + extensions + settings) for repeatable workflows. Rust core gives a single fast binary; sessions are resumable and lead/worker multi-agent orchestration exists.
- **Harness integration notes:** Copy: extensions-as-MCP-only (no privileged built-ins), recipes as portable task packages, per-session extension scoping. Avoid: recipe/config sprawl across YAML formats; the desktop Electron app is a lot of surface area for what the CLI does.

Crush

- **Repo:** <https://github.com/charmbracelet/crush>
- **Stars:** ~28k
- **Language:** Go
- **What it does:** Charm's glamorous terminal coding agent — a Bubble Tea TUI with multi-model sessions (switch models mid-conversation while preserving context), LSP-aware context, MCP support, and session-based state.
- **Why it's the best:** The benchmark for **terminal UX craft**: gorgeous, keyboard-first TUI with diffs, spinners, and dialogs done right, in a single static Go binary. Architecturally notable for clean separation (agent loop, LLM provider abstraction, tools, TUI) and for mixing models per session — e.g. a cheap model for small tasks, a strong one for hard ones — which few harnesses support.
- **Harness integration notes:** Copy: Go single-binary distribution, Bubble Tea component model if building a TUI, mid-session model switching with context carry-over, LSP integration. Avoid: tool set and permission model are thinner than Cline/Aider; treat it as a UX reference, not an agent-loop reference.

Roo Code

- **Repo:** <https://github.com/RooCodeInc/Roo-Code>
- **Stars:** ~24k
- **Language:** TypeScript
- **What it does:** Cline fork that pushed the "whole dev team in your editor" idea: switchable **modes** (Code, Architect, Ask, Debug, plus fully custom modes) each with its own system prompt, tool restrictions, and model; subtask orchestration (boomerang tasks); roomodes/ `.roo` rule directories.

- **Why it's the best:** Canonical reference for **mode/persona engineering**: a mode = (system prompt, allowed tool groups, preferred model, file-regex edit restrictions), which is a simple, powerful way to get specialized behavior (architect can't edit code, only plan) out of one agent loop. Its `.roo/rules-<mode>/` directories show how to ship per-mode workflow instructions as versioned files.
- **Harness integration notes:** Copy: modes as prompt+toolset+model triples, tool-group gating per mode, orchestrator mode delegating subtasks with isolated context. Avoid: feature accretion — Roo shipped everything the community asked for and the codebase/settings UI paid for it; note the README disclaimer that Roo Code, Inc shut down the extension in May 2026 (community forks continue), so treat it as a design document, not a dependency.

SWE-agent

- **Repo:** <https://github.com/SWE-agent/SWE-agent>
- **Stars:** ~20k
- **Language:** Python
- **What it does:** Princeton/Stanford research agent (NeurIPS 2024) that takes a GitHub issue and fixes it autonomously; the project that defined the **Agent-Computer Interface (ACI)** concept — the insight that tool design for LMs matters as much as the model.
- **Why it's the best:** The minimal, readable reference agent loop. Its ACI findings are directly load-bearing: custom `str_replace_editor`-style file viewer with line ranges and edit linting, constrained shell commands, search tools that return bounded results, and guardrails (e.g. rejecting edits with syntax errors before applying) measurably raise solve rates. Trajectory logging format is an eval gold standard.
- **Harness integration notes:** Copy: design tools *for the model* (bounded outputs, one obvious way to do each thing, immediate validation feedback), keep the loop small enough to read in an afternoon; use its trajectory schema for evals. Avoid: it's benchmark-oriented (SWE-bench), not a daily-driver UX — no approvals, no checkpoints, minimal multi-turn human interaction.

Honorable Mentions

- **Continue** (continuedev/continue, ~28k) — VS Code/JetBrains open-source autopilot; good reference for IDE diff/apply streaming and config-as-code (`config.yaml` with blocks), less agentic than Cline.
- **plandex** (plandex-ai/plandex, ~14k) — terminal agent built around versioned, branchable "plans" applied across many files; worth studying for large-change sandboxing, but development has slowed.
- **Mentat** (AbanteAI/mentat, ~3k) — early terminal agent whose context-auto-inclusion and "code map" ideas influenced later tools; mostly superseded, historically interesting.
- **Claude-Engineer-style demos** (e.g. Doriandarko/claude-engineer) — small single-file agent loops; useful as 500-line illustrations of tool-use loops, not as architecture.

2 LLM Memory & Knowledge

Persistent memory is what separates a stateless chatbot from a true agent: a world-class harness needs layered memory (working context, episodic history, semantic facts, and relational knowledge) that survives sessions, compresses over time, and is retrievable with low latency. This report covers the best open-source projects for agent memory libraries, memory servers (including MCP), temporal knowledge graphs, and the vector stores that underpin them.

mem0

- **Repo:** <https://github.com/mem0ai/mem0>
- **Stars:** ~65.5k
- **Language:** Python (plus `mem0-ts` TypeScript SDK)
- **What it does:** Drop-in memory layer for AI agents and apps. Extracts salient facts from conversations with an LLM, deduplicates/updates them, and stores them in vector + optional graph stores for later retrieval. Ships a Python library, a REST server, and plugins for Claude/Cursor/Codex/Kimi.
- **Why it's the best:** The most popular dedicated agent-memory library by a wide margin, with published benchmarks (LOCOMO) showing strong accuracy/latency tradeoffs vs. full-context baselines. Pluggable vector stores (Qdrant, pgvector, etc.), graph memory variant, and both library-level and server-level deployment.
- **Harness integration notes:** Embed as a library (`Memory.from_config(...)`) behind your harness's memory interface, or run its FastAPI server as a sidecar; its fact-extraction + update/delete reconciliation loop is the reference design to copy even if you roll your own.

Letta (formerly MemGPT)

- **Repo:** <https://github.com/letta-ai/letta>
- **Stars:** ~24.8k
- **Language:** Python / TypeScript (active source now in `letta-ai/letta-code`)
- **What it does:** Platform for stateful agents whose memory is a first-class, self-editing resource: agents manage their own core memory blocks and archival storage via tool calls, with an agent server, ADE, SDK, and channels (Slack/Telegram/Discord).
- **Why it's the best:** Originates the MemGPT "OS-style" memory model — tiered context with agent-directed memory edits — which is the most influential memory architecture in the field. Full server + SDK + multi-channel runtime makes it the most complete stateful-agent platform.
- **Harness integration notes:** Even if you don't adopt the server, study its memory-block/tool-call design: giving the agent `core_memory_replace` / `archival_search` style tools is a proven pattern. The Letta Agent SDK can also be embedded directly for memory-managed agents.

Zep

- **Repo:** <https://github.com/getzep/zep>
- **Stars:** ~4.9k
- **Language:** Go (server; Python/TS clients)
- **What it does:** Memory layer server for agents built on a temporal knowledge graph: ingests chat history and business data, extracts entities/facts with validity intervals, and serves hybrid (semantic + BM25 + graph) search. Repo also hosts examples, integrations, an ontology system, and an official MCP server (`mcp/zep-mcp-server`).
- **Why it's the best:** Best-in-class temporal reasoning over agent memory — facts carry `valid_at / invalid_at` so agents can answer "what was true then." Benchmarks (DMR, LongMemEval) show it beating baseline memory approaches; the graph model scales to enterprise data, not just chat.
- **Harness integration notes:** Run as a sidecar memory service and call it from the harness over its API or MCP server; its temporal-edge model and hybrid retrieval pipeline are the blueprint for production-grade agent recall.

Graphiti

- **Repo:** <https://github.com/getzep/graphiti>
- **Stars:** ~31k
- **Language:** Python
- **What it does:** Library (plus `mcp_server` and REST `server`) for building real-time, temporally-aware knowledge graphs for agents. Incrementally ingests episodes, extracts entities/relationships, resolves duplicates, and supports hybrid search over the graph (Neo4j, FalkorDB, Neptune backends).
- **Why it's the best:** The leading open-source library for agent knowledge graphs: real-time incremental updates (no batch recomputation), bi-temporal modeling, and custom entity-type ontologies via Pydantic. Powers Zep's memory engine and has a first-party MCP server.
- **Harness integration notes:** Use `graphiti-core` directly as the harness's structured-memory component, or mount its MCP server to give any agent graph memory; ideal when relational/temporal queries matter more than pure vector similarity.

cognee

- **Repo:** <https://github.com/topoteretes/cognee>
- **Stars:** ~30.8k
- **Language:** Python
- **What it does:** Memory/knowledge engine that turns documents, chats, and structured data into a combined knowledge graph + vector store ("ECL" pipelines: extract, cognify, load). Provides search over graph and embeddings, with many LLM, vector DB, and graph DB backends.
- **Why it's the best:** Strongest open-source "memory for agents in 5 lines" story that unifies RAG-style ingestion with graph construction; actively developed in 2025-2026 with benchmarks showing

gains over naive RAG. Backend-agnostic (Qdrant, LanceDB, pgvector, Neo4j, Kùzu, etc.).

- **Harness integration notes:** Good fit as the harness's long-term semantic memory over files/repos: `cognee.add()` + `cognee.cognify()` + `cognee.search()`. Its pipeline abstraction is a clean model for pluggable memory extraction/compression stages.

Official MCP Servers (memory server)

- **Repo:** <https://github.com/modelcontextprotocol/servers>
- **Stars:** ~90.4k
- **Language:** TypeScript / Python
- **What it does:** The reference collection of Model Context Protocol servers, including the official `@modelcontextprotocol/server-memory`: a knowledge-graph-based persistent memory server (entities, relations, observations stored as JSONL) that any MCP-capable agent can call.
- **Why it's the best:** The canonical, protocol-native way to bolt persistent memory onto any MCP-compatible harness with zero custom integration; the memory server is the de-facto reference implementation for KG-style agent memory tools.
- **Harness integration notes:** If your harness speaks MCP, you get memory for free by launching this server; its tool schema (`create_entities` , `add_observations` , `search_nodes` , ...) is a minimal, well-tested contract worth mirroring in native memory tools.

LanceDB

- **Repo:** <https://github.com/lancedb/lancedb>
- **Stars:** ~11.4k
- **Language:** Rust (with Python/TypeScript APIs)
- **What it does:** Embedded, serverless vector database built on the Lance columnar format. Stores embeddings alongside the raw data (text, images), supports hybrid search (vector + FTS via Tantivy), SQL-style filtering, and zero-copy versioning on disk or object storage.
- **Why it's the best:** Best embedded vector store for agent memory: no server to run, multi-modal by design, and cheap enough to keep per-agent memory files. Columnar versioning makes memory snapshots/time-travel natural.
- **Harness integration notes:** Ideal default local memory store for a harness — one Lance table per agent/session with metadata columns for timestamps and memory type; pairs well with any extraction layer above (mem0, cognee both support it).

pgvector

- **Repo:** <https://github.com/pgvector/pgvector>
- **Stars:** ~23k
- **Language:** C (PostgreSQL extension)
- **What it does:** Adds vector similarity search (HNSW and IVFFlat indexes) to PostgreSQL, so embeddings live next to relational data with full SQL, transactions, and filtering.

- **Why it's the best:** The pragmatic production choice: most deployments already run Postgres, and pgvector removes an entire infrastructure component while keeping ACID guarantees over memory + metadata. Mature, actively maintained, and supported by every major memory framework (mem0, Letta, LangChain).
- **Harness integration notes:** Use when the harness already persists state in Postgres — one `embedding vector(N)` column plus HNSW index gives durable, queryable agent memory; combine with row-level timestamps for recency-weighted recall.

Honorable Mentions

- Qdrant (<https://github.com/qdrant/qdrant>, ~34.6k) — top dedicated vector DB server (Rust); best when memory must scale beyond a single node.
- Chroma (<https://github.com/chroma-core/chroma>, ~29.3k) — popular embedded/hosted vector DB; the default in many tutorials, easy local memory.
- Weaviate (<https://github.com/weaviate/weaviate>, ~12k) — vector DB with built-in hybrid search and GraphQL; strong for managed knowledge retrieval.
- MemGPT archive branch (<https://github.com/letta-ai/letta/tree/archive>) — original research codebase for the tiered-memory paper; historically important reading.
- Memobase (<https://github.com/memodb-io/memobase>) — profile-based user memory backend; rising alternative to mem0 for user-centric agents.

3 Token Efficiency & Context Compression

Every token an agent sends costs money and latency, and long agent sessions blow past context windows fast. A world-class harness needs tools at every layer: compressing prompts before they leave, counting and budgeting tokens accurately, caching repeated work, keeping the KV cache small on self-hosted models, serializing structured data in token-efficient formats, and engineering what stays in context across turns. These are the best open-source projects for each layer.

LLMLingua (Microsoft)

- **Repo:** <https://github.com/microsoft/LLMLingua>
- **Stars:** ~7k
- **Language:** Python
- **What it does:** Prompt-compression library family — LLMLingua, LongLLMLingua (long-context, question-aware), and LLMLingua-2 (distilled, task-agnostic) — that drops low-information tokens from prompts using a small LM, achieving up to 20x compression with minimal accuracy loss.
- **Why it's the best:** The reference implementation for learned prompt compression, from Microsoft Research, with published benchmarks and pre-trained compressor models on Hugging Face. LLMLingua-2 is fast enough (bidirectional small encoder) to sit inline in a request path.
- **Harness integration notes:** Wrap retrieved documents and tool outputs in a compression step before appending them to the message list; LongLLMLingua's question-aware mode preserves the parts of a document relevant to the current task. Also usable purely as a design reference for a "compress before you truncate" pipeline.

tiktoken (OpenAI)

- **Repo:** <https://github.com/openai/tiktoken>
- **Stars:** ~16k
- **Language:** Rust (Python bindings)
- **What it does:** Fast BPE tokenizer library for OpenAI models (cl100k_base, o200k_base, etc.), 3-6x faster than comparable Python tokenizers, used to count tokens before sending requests.
- **Why it's the best:** The de facto standard for token counting; exact parity with what OpenAI bills, and the Rust core makes counting effectively free even on large corpora. Encoding specs are public and reusable.
- **Harness integration notes:** Use for per-message token accounting, context-window budgeting, and truncation decisions. For non-OpenAI models, pair with `huggingface/tokenizers` (~10k stars) behind a common `count_tokens(text, model)` abstraction so the harness stays model-agnostic.

GPTCache (Zilliz)

- **Repo:** <https://github.com/zilliztech/GPTCache>
- **Stars:** ~8k

- **Language:** Python
- **What it does:** Semantic cache for LLM queries — stores past prompts/responses and answers new queries from cache using embedding similarity, with pluggable vector stores (Milvus, FAISS, Chroma), eviction policies, and cache-hit evaluation.
- **Why it's the best:** The most complete open-source LLM caching library: semantic (not just exact-match) lookup, modular data managers, similarity thresholds, and adapters for OpenAI/LangChain. Directly cuts both cost and latency on repetitive agent workloads.
- **Harness integration notes:** Insert as a middleware layer in front of the model client; cache tool-call results and repeated sub-queries (e.g. identical file reads or web fetches across agent steps). Its evaluator/similarity-threshold design is worth copying even if you build your own cache.

vLLM

- **Repo:** <https://github.com/vllm-project/vllm>
- **Stars:** ~60k
- **Language:** Python (CUDA/C++ kernels)
- **What it does:** High-throughput LLM serving engine built around PagedAttention — OS-style paged KV-cache management that eliminates memory fragmentation — plus continuous batching, prefix caching, and chunked prefill.
- **Why it's the best:** The industry-standard inference server; PagedAttention (SOSP'23) is the foundational technique for KV-cache efficiency, and its automatic prefix caching (APC) reuses KV blocks for shared prompt prefixes — exactly what multi-turn agents with long stable system prompts need.
- **Harness integration notes:** If the harness can self-host models, run them behind vLLM and structure prompts so the invariant prefix (system prompt, tool schemas) comes first to maximize prefix-cache hits. Study its block manager and prefix-cache eviction (LRU over KV blocks) as the blueprint for any cache design.

SGLang

- **Repo:** <https://github.com/sgl-project/sglang>
- **Stars:** ~18k
- **Language:** Python
- **What it does:** Serving engine and programming framework for LLMs featuring RadixAttention — a radix-tree over KV caches that automatically shares prefixes across requests — plus a structured-generation frontend and cache-aware scheduling.
- **Why it's the best:** RadixAttention is the state of the art for prefix reuse in agentic workloads: tree-of-calls, few-shot prompts, and multi-agent branches all share KV automatically, giving up to ~5x throughput on shared-prefix workloads. Very actively developed in 2025-2026 with strong benchmark results vs. vLLM.
- **Harness integration notes:** For tree-search agents or parallel branches off a common conversation, SGLang's radix tree means branches cost almost no extra prefill. Its cache-aware router is the model for routing requests to the worker holding the warmest prefix.

NVIDIA KVPress

- **Repo:** <https://github.com/NVIDIA/kvpress>
- **Stars:** ~1.2k
- **Language:** Python
- **What it does:** Library of 20+ KV-cache compression methods (SnapKV, H2O, StreamingLLM, TOVA, PyramidKV, and more) implemented as Hugging Face `transformers` pipeline hooks, plus an evaluation suite on long-context benchmarks.
- **Why it's the best:** The broadest, best-maintained collection of KV-compression algorithms under one uniform interface, from NVIDIA, with reproducible benchmark harness (RULER, LongBench, Needle-in-a-Haystack). Lets you compare eviction policies without reimplementing papers.
- **Harness integration notes:** When running local models for long agent sessions, attach a press (e.g. SnapKV or StreamingLLM-style sink+window) to cap KV memory as conversation grows. The paper-implementations-as-plugins architecture is a good template for swappable compression strategies.

TOON (Token-Oriented Object Notation)

- **Repo:** <https://github.com/toon-format/toon>
- **Stars:** ~25k
- **Language:** TypeScript (Python port: `toon-format/toon-python`)
- **What it does:** A compact, human-readable serialization format for JSON data in LLM prompts — declares array fields once in a YAML-like header, then streams rows CSV-style — cutting token usage ~30-60% vs. JSON on uniform tabular data. Ships with spec, SDK, CLI, and benchmarks.
- **Why it's the best:** The breakout 2025 standard for token-efficient structured data; lossless round-trip to JSON, tokenizer-aware benchmarks across GPT/Claude/Llama tokenizers, and rapidly growing multi-language ecosystem. Exploded to tens of thousands of stars within months of release.
- **Harness integration notes:** Serialize tool outputs, RAG results, and structured state as TOON instead of JSON when feeding them into prompts; keep JSON only at API boundaries. Its benchmark suite is a template for measuring "tokens per record" across formats in your own harness.

Selective Context

- **Repo:** https://github.com/liyucheng09/Selective_Context
- **Stars:** ~1k
- **Language:** Python
- **What it does:** Compresses context by measuring token/phrase self-information (surprisal) with a small LM and dropping low-information content — deduplicating and pruning sentences, phrases, or tokens while preserving meaning.
- **Why it's the best:** Historically important as the information-theoretic baseline for prompt compression (EMNLP 2023); simple, model-agnostic, and complementary to LLMLingua — its lexical-level filtering composes well with other compressors.

- **Harness integration notes:** Useful as a lightweight, dependency-minimal pre-filter for verbose tool output (logs, docs) before more expensive compression; also a good reference for surprisal-based salience scoring when ranking what to keep in a context window.

Honorable Mentions

- huggingface/tokenizers — Rust tokenizer library; the model-agnostic counterpart to tiktoken for counting across Llama/Qwen/Mistral vocabularies.
- LMCache (LMCache/LMCache) — KV-cache offloading/sharing layer (GPU→CPU→disk) that pairs with vLLM for cross-session prefix reuse.
- StreamingLLM (mit-han-lab/streaming-llm) — attention-sink + sliding-window technique enabling effectively infinite streams; conceptually foundational for session-length KV management.
- Anthropic-style context editing / compaction — not a repo, but the "summarize-and-truncate oldest turns" pattern (implemented in Claude Code and OpenHands) is the pragmatic baseline for agent context-window management.
- prompt-reddit/compression toolkits like 3DAgentWorld/Toolkit-for-Prompt-Compression — curated collections of compression papers/implementations for surveying the field.

4 Web Scraping & Crawling for LLMs

Web access is the single most-requested tool capability for agent harnesses: agents need to fetch pages, convert noisy HTML into token-efficient markdown, survive anti-bot defenses, and search the web. This category covers the best open-source building blocks — from LLM-native crawlers to battle-tested scraping frameworks and self-hostable search — that a harness can embed directly as tools.

Crawl4AI

- **Repo:** <https://github.com/unclecode/crawl4ai>
- **Stars:** ~84k
- **Language:** Python
- **What it does:** Open-source, LLM-friendly async web crawler and scraper built on Playwright. Produces clean markdown, structured JSON (via CSS/XPath/LLM extraction), screenshots, and PDFs, with deep-crawling strategies, proxy support, and a Docker-deployable API/MCP server.
- **Why it's the best:** The most popular purpose-built "crawl for LLMs" library — its markdown generation is tuned for token efficiency (fit-markdown via content filtering), and it ships BM25-based content pruning, chunking strategies, and extraction schemas out of the box. Extremely active (1,600+ commits, frequent 2025-2026 releases).
- **Harness integration notes:** Embed as a Python tool (`AsyncWebCrawler.arun(url)`) or run its Docker service and call the REST/MCP endpoint; its `BrowserConfig` / `CrawlerRunConfig` pattern is a good model for harness-level crawl configuration objects.

Firecrawl

- **Repo:** <https://github.com/firecrawl/firecrawl>
- **Stars:** ~182k
- **Language:** TypeScript
- **What it does:** Web data API that turns URLs into LLM-ready markdown or structured data: scrape, crawl entire sites, map URLs, search, and agent-driven "interact" actions (click, type, scroll) — self-hostable with a hosted cloud option.
- **Why it's the best:** The most-starred project in the category; its `/scrape`, `/crawl`, `/map`, and `/extract` endpoints define the de-facto API shape for LLM web access, with anti-bot handling, JS rendering, and LLM extraction (schema → JSON) built in. Very active development and broad SDK coverage (JS/Python).
- **Harness integration notes:** Self-host the API service and expose it to agents as a single `web_scrape` / `web_crawl` tool; its job-queue architecture (Redis + workers) is a solid reference for scaling a harness's fetch layer.

trafilatura

- **Repo:** <https://github.com/adbar/trafilatura>

- **Stars:** ~6.8k
- **Language:** Python
- **What it does:** Fast, dependency-light library for web text extraction and metadata harvesting — downloads pages, strips boilerplate, and outputs clean text/markdown/XML with title, author, date, and comments.
- **Why it's the best:** Best-in-class accuracy on main-content extraction benchmarks (comparing favorably to readability/newspaper3k), no headless browser needed, and very fast — ideal as the cheap first-pass before escalating to a browser. Actively maintained with regular releases.
- **Harness integration notes:** Use as the default "static fetch + extract" path in a tiered fetch strategy: trafilatura first, Playwright fallback only when JS rendering is required. Its extraction-only mode works on HTML your harness already fetched.

Jina Reader (reader)

- **Repo:** <https://github.com/jina-ai/reader>
- **Stars:** ~12k
- **Language:** TypeScript
- **What it does:** The engine behind r.jina.ai — converts any URL into clean, LLM-friendly markdown (and search grounding via s.jina.ai), handling JS-heavy pages, PDFs, and images with captioning.
- **Why it's the best:** Popularized the "prefix a URL to get markdown" pattern that many harnesses now copy; its readability pipeline plus Puppeteer rendering is a proven, simple architecture for a URL→markdown service. Self-hostable with Docker.
- **Harness integration notes:** Either call the free hosted endpoint (r.jina.ai) for zero-infra web reads, or self-host and study its readability + snapshot pipeline as a template for your own reader service.

Playwright

- **Repo:** <https://github.com/microsoft/playwright>
- **Stars:** ~96k
- **Language:** TypeScript (with Python/.NET/Java ports)
- **What it does:** Cross-browser automation framework for Chromium, Firefox, and WebKit with auto-waiting, network interception, isolated contexts, and first-class headless support.
- **Why it's the best:** The industry standard for headless browser control — more reliable than Puppeteer for scraping due to auto-wait, strict selectors, and context isolation; nearly every scraping/agent tool above (Crawl4AI, Firecrawl, browser-use) is built on it. Backed by Microsoft, very actively maintained.
- **Harness integration notes:** This is the browser substrate for any harness that renders JS pages, takes screenshots, or gives agents interactive browsing. Use its Python or TS library directly and layer stealth plugins (e.g. rebrowser patches) for anti-bot resilience.

Scrapy

- **Repo:** <https://github.com/scrapy/scrapy>

- **Stars:** ~64k
- **Language:** Python
- **What it does:** The venerable, production-grade web crawling framework: spiders, middlewares, pipelines, concurrency/throttling, retries, and export feeds — plus the scrapy-playwright plugin for JS pages.
- **Why it's the best:** Unmatched for large-scale, structured crawling jobs (thousands of pages with politeness rules, dedup, and pipelines); its middleware architecture is the canonical design for request/response processing chains. Mature and still actively maintained.
- **Harness integration notes:** Overkill for single-page agent fetches, but the right tool when a harness needs scheduled bulk ingestion (e.g. building a knowledge base from a whole site). Borrow its middleware/pipeline patterns for your own fetch layer.

SearXNG

- **Repo:** <https://github.com/searxng/searxng>
- **Stars:** ~37k
- **Language:** Python
- **What it does:** Self-hostable, privacy-respecting metasearch engine aggregating 200+ search engines (Google, Bing, DuckDuckGo, Brave, Wikipedia, arXiv...) with a JSON API.
- **Why it's the best:** The de-facto open-source answer to "give my agent web search without paying Tavily/SerpAPI" — free, unlimited, and self-hosted, with per-engine configuration and result merging. Very active community and releases.
- **Harness integration notes:** Run a private instance and expose its `/?q=...&format=json` endpoint as the harness's `web_search` tool; pair results with trafilatura/Crawl4AI fetches for a complete search→read pipeline.

ScrapeGraphAI

- **Repo:** <https://github.com/ScrapeGraphAI/Scrapegraph-ai>
- **Stars:** ~31k
- **Language:** Python
- **What it does:** LLM-powered scraping library: describe what you want in natural language (or a Pydantic schema) and its graph pipelines (SmartScraper, SearchGraph, ScriptCreator) extract structured data from pages or whole sites.
- **Why it's the best:** The leading open-source project for LLM-driven structured extraction — turns "get me all speakers and titles from this page" into validated JSON without writing selectors, and supports many LLM backends (OpenAI, Anthropic, Ollama, etc.).
- **Harness integration notes:** Embed as the harness's `extract_structured_data(url, schema)` tool on top of your existing fetch layer; its graph-of-nodes design (fetch → parse → LLM-extract → validate) is a clean pattern to replicate.

Honorable Mentions

- **undetected-chromedriver** (ultrafunkamsterdam/undetected-chromedriver, ~13k) — patched Selenium ChromeDriver that evades common anti-bot detection; useful when Selenium is required, though Playwright + stealth is the modern path.
- **python-markdownify** (matthewwithanm/python-markdownify, ~2.2k) — tiny, dependable HTML→markdown converter; good lightweight default when you already have cleaned HTML.
- **duckduckgo_search** (deedy5/duckduckgo_search, ~3k) — zero-infra DDG search library for quick agent search prototypes; rate-limited, so graduate to SearXNG for production.
- **newspaper4k** (AndyTheFactory/newspaper4k) — maintained fork of newspaper3k for article extraction; simpler but less accurate than trafilatura.
- **Camoufox** (daijro/camoufox) — anti-detect Firefox build with a Playwright-compatible interface; rising star for stealth browsing.

5 Mass File Search & Code Navigation

<p>An agent harness lives or dies by how fast and precisely it can locate code. LLMs cannot hold a large repo in context, so the harness must shell out to best-in-class search and indexing tools: literal/regex search for exact matches, structural (AST) search for syntactic patterns, symbol indexing for repo maps, and trigram indexes for instant search at scale. The projects below are the tools a world-class harness should embed or shell out to.</p>

ripgrep

- **Repo:** <https://github.com/BurntSushi/ripgrep>
- **Stars:** ~68k
- **Language:** Rust
- **What it does:** Recursively searches directories for regex patterns while respecting .gitignore, hidden-file rules, and binary detection. The de-facto fastest line-oriented search tool.
- **Why it's the best:** SIMD-accelerated literal optimizations, automatic parallelism, and a finite-automata regex engine make it 5-50x faster than GNU grep/ag in benchmarks. Its `--json` output mode gives machine-parseable match events (begin/match/end with byte offsets) designed for programmatic consumers.
- **Harness integration notes:** Shell out with `rg --json` and stream-parse match events; this is the standard "grep tool" in Claude Code, Aider, and most agent harnesses. Respect its ignore semantics so agents never search node_modules or build output.

fd

- **Repo:** <https://github.com/sharkdp/fd>
- **Stars:** ~45k
- **Language:** Rust
- **What it does:** A fast, user-friendly `find` alternative for file discovery with sensible defaults (gitignore-aware, hidden files skipped, parallel traversal).
- **Why it's the best:** Consistently the fastest directory walker for file-name queries; simple glob syntax, regex support, and `--exec / --exec-batch` for command batching. Same ignore-file semantics as ripgrep, so file discovery and content search stay consistent.
- **Harness integration notes:** Use for the harness's "list files" / "glob" tool (`fd --type f --full-path`). Pairing fd for paths + ripgrep for content covers nearly all lexical navigation an agent needs.

fzf

- **Repo:** <https://github.com/junegunn/fzf>
- **Stars:** ~83k
- **Language:** Go

- **What it does:** A general-purpose command-line fuzzy finder; also embeddable as a library (junegunn/fzf's algorithm is extracted in `github.com/junegunn/fzf/src/algo`) implementing the FuzzyMatchV2 scoring algorithm.
- **Why it's the best:** Its scoring algorithm (character bonuses for word boundaries, camelCase, consecutive matches) is the reference implementation ported to many languages (fzf-lua, fzy alternatives). Extremely fast over millions of entries with a streaming architecture.
- **Harness integration notes:** For an interactive harness UI, fzf is the file/symbol picker of choice; for headless agents, port or bind the FuzzyMatchV2 scoring to rank file paths and symbol names when an agent gives an approximate identifier.

ast-grep

- **Repo:** <https://github.com/ast-grep/ast-grep>
- **Stars:** ~16k
- **Language:** Rust
- **What it does:** A CLI and library (plus napi bindings) for structural code search, lint, and rewriting based on tree-sitter ASTs, using a pattern language with `$META` variables.
- **Why it's the best:** Combines tree-sitter's multi-language parsing (20+ languages) with a grep-like UX: `sg -p 'foo($A)'` finds syntactic matches regardless of formatting. Supports YAML rules, relational constraints (inside/has/follows), and codemod rewriting with `--rewrite`. Much faster than Semgrep for pure syntactic queries.
- **Harness integration notes:** Ideal "structural search" tool for agents — finds all call sites, JSX components, or patterns regex can't express safely. The napi/pyo3 bindings let a harness embed it in-process; the CLI's `--json` mode suits shell-out. Also the engine behind this harness's own `ast_edit` tool.

Semgrep

- **Repo:** <https://github.com/semgrep/semgrep>
- **Stars:** ~17k
- **Language:** OCaml (with Python CLI wrapper)
- **What it does:** Lightweight static analysis with a code-pattern syntax that looks like the target language; supports taint tracking, dataflow, and cross-file analysis (Semgrep Code/Pro) across 30+ languages.
- **Why it's the best:** The most expressive open-source pattern engine for semantic-ish queries: metavariables, typed metavariables, pattern-inside/pattern-not combinators, and constant propagation. Huge community rule registry (`semgrep.dev/rules`) for security and correctness patterns.
- **Harness integration notes:** Use `semgrep --json` for deep, semantics-aware queries ("find SQL built from string concat reaching `execute()`") that ast-grep can't express. Heavier than ast-grep — reserve for security audits and cross-function dataflow, not routine navigation.

zoekt

- **Repo:** <https://github.com/sourcegraph/zoekt>
- **Stars:** ~2k
- **Language:** Go
- **What it does:** A trigram-based full-text code search engine (originally by Google, maintained by Sourcegraph) that indexes repos and answers substring/regex queries in milliseconds over millions of files.
- **Why it's the best:** The indexing engine that powers Sourcegraph's search; incremental indexing, shard-based design, symbol extraction via ctags/tree-sitter, and a query language with file/content/repo filters. Nothing else open-source matches its query latency at scale.
- **Harness integration notes:** For harnesses working across many repos or monorepos, run zoekt-indexserver + query via its API instead of repeated ripgrep scans — turns multi-second greps into ~10ms lookups. Its trigram-index design doc is also the blueprint if you build your own index.

tree-sitter

- **Repo:** <https://github.com/tree-sitter/tree-sitter>
- **Stars:** ~27k
- **Language:** Rust/C
- **What it does:** An incremental parser generator and runtime with grammars for 100+ languages; produces concrete syntax trees with precise byte ranges, plus a query language (S-expressions with captures) for extracting nodes.
- **Why it's the best:** The foundation of modern code intelligence: error-tolerant (parses incomplete/broken code, crucial for LLM-generated snippets), incremental (re-parse only edited ranges), and bindings for every major language. Powers GitHub's code navigation, Neovim, Zed, ast-grep, and Aider's repo-map.
- **Harness integration notes:** Embed tree-sitter directly for symbol extraction, outline generation, and syntax-aware chunking. Use its query language to build a ctags-style tag index: capture `@definition.function`, `@reference.call`, etc., to feed repo maps and "where is X defined" agent tools.

Aider (repo-map)

- **Repo:** <https://github.com/Aider-AI/aider>
- **Stars:** ~49k
- **Language:** Python
- **What it does:** An AI pair-programming CLI whose `repomap` module builds a token-budgeted map of the repository: tree-sitter extracts definitions/references per file, then a PageRank-style ranking over the reference graph selects the most important symbols to show the LLM.
- **Why it's the best:** The repo-map is the most influential, battle-tested technique for giving an LLM repo-wide awareness within a token budget — it compresses thousands of files into a ranked

skeleton of key signatures. Aider's published benchmarks show repo-maps materially improve editing accuracy.

- **Harness integration notes:** Read `aider/repomap.py` as the reference implementation: tree-sitter tag extraction → networkx PageRank over (file, identifier) edges → greedy token-budget packing. Reimplement this pattern in your harness; it's the single highest-leverage context-engineering trick for code navigation.

Honorable Mentions

- **comby** (<https://github.com/comby-tools/comby>, ~2.7k) — language-agnostic structural matching with `:[hole]` syntax; great for lightweight multi-language rewrites without full ASTs, though less actively maintained than ast-grep.
- **universal-ctags** (<https://github.com/universal-ctags/ctags>, ~7k) — maintained ctags with many parsers; still the simplest way to emit tag files, but tree-sitter queries generally supersede it for new harnesses.
- **Sourcegraph** (<https://github.com/sourcegraph/sourcegraph>) — full code-search platform built on zoekt; heavy to self-host but its query language and SCIP-based precise code intel are worth studying.
- **livegrep** (<https://github.com/livegrep/livegrep>) — regex search over an indexed corpus using suffix automata; an alternative to zoekt with a nice web UI.
- **grep / silver searcher (ag)** — historically important, but both are effectively superseded by ripgrep.
- **SCIP / LSIF indexers** (<https://github.com/sourcegraph/scip>) — compiler-grade precise code-intel index format; the right choice when a harness needs true go-to-definition rather than textual heuristics.

6 Browser Automation & Computer Use for Agents

Giving an LLM harness eyes and hands on the web — and on the desktop — is the single highest-leverage capability upgrade after code execution. This category covers the agent frameworks that drive browsers (DOM/accessibility-tree agents, vision agents, hybrid SDKs), the browser infrastructure layer they run on, computer-use/desktop control, and the benchmarks used to measure it all. The projects below are the current best-in-class open-source options, verified against their GitHub repos in September 2026.

browser-use

- **Repo:** <https://github.com/browser-use/browser-use>
- **Stars:** ~115k
- **Language:** Python (also ships a TypeScript SDK)
- **What it does:** The de-facto standard LLM browser agent. Serializes the DOM into an indexed, LLM-friendly element list, pairs it with screenshots, and runs a tool-calling loop over Playwright/CDP with watchdogs for CAPTCHAs, popups, downloads, and session recording.
- **Why it's the best:** Largest community in the space by an order of magnitude, extremely active (multiple releases per month, its own BU Bench v2 with per-model cost/score curves), and battle-tested against real sites. Its DOM-to-prompt representation (interactive-element indexing with stable element IDs) has become the reference design most other agents copy.
- **Harness integration notes:** Embed as a library (`Agent(task=..., llm=...)`) or adopt its two core ideas directly: (1) the indexed-DOM serialization + screenshot multimodal observation, and (2) the watchdog pattern for handling CAPTCHAs/dialogs/downloads as first-class events. Its system prompts under `browser_use/agent/system_prompts/` are worth studying verbatim. MIT licensed.

Stagehand

- **Repo:** <https://github.com/browserbase/stagehand>
- **Stars:** ~24k
- **Language:** TypeScript
- **What it does:** Browserbase's open-source AI browser SDK. Adds three primitives — `act` (natural-language actions), `extract` (schema-typed data extraction), and `observe` (plan candidate actions) — plus an `agent` mode, on top of Playwright.
- **Why it's the best:** The cleanest hybrid deterministic/LLM design: you write normal Playwright code and drop to natural language only where selectors are brittle, with action caching so repeated runs skip the LLM call. Best-in-class for TypeScript harnesses and for teams that want reliability of code with LLM fallback rather than a fully autonomous loop.
- **Harness integration notes:** The `act / extract / observe` triad is an excellent tool surface to copy for a harness browser tool — especially `observe` returning cached, replayable actions. MIT licensed; works against any Chromium via CDP, not just Browserbase cloud.

Skyvern

- **Repo:** <https://github.com/Skyvern-AI/skyvern>
- **Stars:** ~23k
- **Language:** Python
- **What it does:** Vision-LLM-driven browser workflow automation. A swarm of agents (planning, element grounding, validation) navigates sites it has never seen, with a no-code workflow builder, credential store (Bitwarden/1Password), TOTP, proxies, and a Playwright-compatible SDK (`page.act`, `page.extract`, `page.validate`, `page.agent.run_task`).
- **Why it's the best:** Strongest production-oriented feature set: persistent browser sessions, webhook callbacks, artifact/download handling, 2FA and credential management, and a published 85.8% WebVoyager result for Skyvern 2.0. The "AI-augmented Playwright" mode (every Playwright action accepts an optional `prompt=` for element location) is a genuinely novel middle ground.
- **Harness integration notes:** AGPLv3 — fine for research/self-hosting, but check licensing before embedding in a commercial harness. Architecturally, study its multi-agent decomposition (separate LLM calls for grounding vs. planning vs. validation) and its per-step artifact recording for debugging agent runs.

Steel Browser

- **Repo:** <https://github.com/steel-dev/steel-browser>
- **Stars:** ~7.7k
- **Language:** TypeScript (Node.js)
- **What it does:** Open-source browser API / sandbox for AI agents — a self-hostable alternative to Browserbase. REST + WebSocket API over containerized Chrome with session management, stealth/fingerprinting, proxy support, CAPTCHA hooks, live-view debugging UI, and file downloads.
- **Why it's the best:** The leading open answer to "where do my agent's browsers actually run?" Batteries-included: sessions, contexts, recordings, and anti-detection in one Docker image, and it speaks standard CDP/Playwright/Puppeteer protocols so any agent framework can attach.
- **Harness integration notes:** Run it as your harness's browser backend and connect agent frameworks over CDP — this cleanly separates agent logic from browser lifecycle. Its session/context API (create, persist cookies, release) is a good model if you build your own browser pool.

UI-TARS

- **Repo:** <https://github.com/bytedance/UI-TARS> (desktop app: <https://github.com/bytedance/UI-TARS-desktop>)
- **Stars:** ~11.5k
- **Language:** Python (models on Hugging Face; desktop app in TypeScript)
- **What it does:** ByteDance's native GUI agent model family (UI-TARS-1.5-7B open weights; UI-TARS-2 tech report Sept 2025). A single vision-language model that takes raw screenshots and emits grounded actions (click/type/scroll at absolute coordinates) for desktop, browser, and mobile — no DOM or accessibility tree required.

- **Why it's the best:** State of the art among open computer-use models on OSWorld and ScreenSpot grounding benchmarks, with RL-based reasoning before action. It is the reference implementation for the "native agent" (pixels-in, actions-out) approach, and the desktop app is a usable computer-use agent out of the box.
- **Harness integration notes:** Serve the 7B model via vLLM and drive it with their action-parsing/post-processing code (`README_coordinates.md` documents the coordinate normalization — the easy-to-get-wrong part). Use it as the fallback for non-web surfaces (native apps, canvas, remote desktops) where DOM agents are blind. Apache 2.0.

WebArena / WorkArena / OSWorld (benchmarks)

- **Repo:** <https://github.com/web-arena-x/webarena> (also <https://github.com/ServiceNow/WorkArena>, <https://github.com/xlang-ai/OSWorld>)
- **Stars:** ~1.6k (WebArena), ~1k (WorkArena), ~2k (OSWorld)
- **Language:** Python
- **What it does:** The standard evaluation environments for web and computer-use agents. WebArena: 812 tasks over self-hosted realistic sites (GitLab, Reddit, e-commerce, maps, CMS) with programmatic success checkers. WorkArena: enterprise tasks on ServiceNow. OSWorld: 369 real desktop tasks inside a VM for computer-use agents.
- **Why it's the best:** These three are the benchmarks every serious browser/desktop agent paper and product reports against; WebArena in particular defined the accessibility-tree observation format and the trajectory-based evaluation harness pattern.
- **Harness integration notes:** Even if you never run the full suites, steal their machinery: the accessibility-tree serializer in `browser_env`, the per-task URL/state reset via Docker, and the evaluator design (string match + URL match + program execution against backend state). Run a WebArena subset as your harness's browser regression suite.

Honorable Mentions

- **OpenHands (All-Hands-AI/OpenHands, ~60k)** — full agent harness whose `BrowsingAgent` and BrowserGym integration are a good reference for embedding browsing as one tool among many.
- **nanobrowser (nanobrowser/nanobrowser, ~8k)** — open-source Chrome-extension browser agent; the lightweight in-browser alternative to OpenAI Operator.
- **LaVague (lavague-ai/LaVague, ~6.4k)** — Large Action Model framework; development has slowed but its retriever-augmented action-generation pipeline is still a worthwhile read.
- **Agent-E (EmergenceAI/Agent-E, ~1.3k)** — AutoGen-based web agent with a strong hierarchical planner/executor split and DOM distillation techniques; largely unmaintained but architecturally influential.
- **Playwright (microsoft/playwright, ~75k) / Puppeteer (puppeteer/puppeteer, ~92k)** — the browser automation substrate nearly everything above runs on; Playwright is the default choice for new harnesses.
- **Open Interpreter (OpenInterpreter/open-interpreter, ~60k)** — its OS mode is a pragmatic screenshot+mouse/keyboard computer-use loop worth studying for a minimal desktop tool.

- **self-operating-computer (OthersideAI/self-operating-computer, ~9k)** — early multimodal desktop agent framework; historically important, superseded by UI-TARS-class models.
- **Midscene.js (web-infra-dev/Midscene, ~10k)** — ByteDance's JS browser-automation SDK with visual grounding; the recommended way to run UI-TARS-style models for web tasks.
- **CogAgent (THUDM/CogAgent, ~2k)** — CogVLM-based GUI agent model from Tsinghua/Zhipu; strong academic grounding work for VLM GUI understanding.
- **WebVoyager (MinorJerry/WebVoyager, ~2k)** — the other standard web-agent benchmark (live-web tasks) and a clean GPT-4V agent loop reference.

7 Inference Speed, TTFT & Local Serving

Time-to-first-token and sustained throughput determine whether an agent harness feels snappy or sluggish, and whether local models are viable fallbacks to hosted APIs. This category covers the inference engines, quantization ecosystems, caching layers, and gateways that a world-class harness should embed or learn from — with special attention to Apple Silicon (the user is on an Apple M1), where llama.cpp (Metal) and MLX-LM are the clear leaders.

vLLM

- **Repo:** <https://github.com/vllm-project/vllm>
- **Stars:** ~92k
- **Language:** Python (CUDA/C++ kernels)
- **What it does:** High-throughput, memory-efficient LLM serving engine built around PagedAttention, continuous batching, chunked prefill, and prefix caching, with an OpenAI-compatible server.
- **Why it's the best:** The de facto standard for GPU serving — state-of-the-art throughput, every major quantization (FP8/NVFP4/INT8/GPTQ/AWQ/GGUF), speculative decoding (n-gram, EAGLE, suffix, DFlash), disaggregated prefill/decode, and 200+ model architectures. Extremely active (2000+ contributors, releases through 2026).
- **Harness integration notes:** Run as an OpenAI-compatible sidecar for any GPU box; study its prefix-caching and chunked-prefill design patterns even if you don't embed it. NVIDIA/AMD/TPU-focused — not the right engine for Apple M1.

SGLang

- **Repo:** <https://github.com/sgl-project/sglang>
- **Stars:** ~36k
- **Language:** Python
- **What it does:** High-performance serving framework for LLMs and multimodal models with RadixAttention prefix caching, a zero-overhead CPU scheduler, and prefill-decode disaggregation.
- **Why it's the best:** Frequently beats vLLM on structured-output and agentic workloads (compressed FSM for 3x faster JSON decoding, cache-aware load balancing); day-0 support for frontier models; next-gen speculative decoding (DFlash, Spec V2). Powers 400k+ GPUs in production (xAI, Cursor, etc.).
- **Harness integration notes:** Its RadixAttention tree-based KV reuse is the reference design for prefix caching in agent loops with shared system prompts — directly relevant to minimizing TTFT for multi-turn agent workloads. GPU/TPU only.

llama.cpp

- **Repo:** <https://github.com/ggml-org/llama.cpp>
- **Stars:** ~129k

- **Language:** C/C++
- **What it does:** Dependency-free LLM/VLM inference in C/C++ with the GGUF format, 1.5–8 bit quantization, and backends for Metal, CUDA, Vulkan, SYCL, and CPU SIMD. Ships `llama-server` (OpenAI-compatible) and a built-in web UI.
- **Why it's the best:** The universal local-inference substrate — Apple Silicon is a first-class citizen (ARM NEON, Accelerate, Metal), making it the fastest practical way to run GGUF models on an M1. Massive ecosystem: nearly every quantized model on Hugging Face ships GGUF first.
- **Harness integration notes:** Embed via `llama-server` as a local OpenAI-compatible endpoint, or link `libllama` directly. On the M1, this is the default local engine; GGUF quantization support in your harness should target its format conventions.

MLX-LM

- **Repo:** <https://github.com/ml-explore/mlx-lm>
- **Stars:** ~7k
- **Language:** Python
- **What it does:** Apple's official LLM package on the MLX array framework — text generation, streaming, batch generation, quantization, and LoRA fine-tuning on Apple Silicon, with Hugging Face hub integration and an OpenAI-compatible server (`mlx_lm.server`).
- **Why it's the best:** Purpose-built by Apple for unified-memory Apple Silicon; on M-series chips it often matches or beats `llama.cpp` for prompt processing (TTFT) thanks to Metal-native kernels, and supports prompt caching (`cache_prompt`) plus distributed inference across Macs. Actively maintained with day-0 support for new architectures.
- **Harness integration notes:** The best Python-native embedding option on the M1 — `pip install mlx-lm` and call `stream_generate` in-process, no server hop. Use its prompt-cache API to cut TTFT on repeated agent system prompts.

TensorRT-LLM

- **Repo:** <https://github.com/NVIDIA/TensorRT-LLM>
- **Stars:** ~14.6k
- **Language:** Python/C++ (CUDA)
- **What it does:** NVIDIA's inference library that compiles LLMs into optimized TensorRT engines with in-flight batching, paged attention, FP8/FP4 quantization, and speculative decoding, plus Python/C++ runtimes and a Triton backend.
- **Why it's the best:** The peak-performance option on NVIDIA hardware — hand-tuned kernels (TRTLLM-GEN attention/GEMM) that top MLPerf Inference benchmarks, especially for Blackwell-class GPUs and NVFP4. vLLM and SGLang both consume its kernels.
- **Harness integration notes:** Relevant only if the harness targets NVIDIA deployment; its kernel library and disaggregated-serving architecture are worth studying. Irrelevant on Apple M1.

Ollama

- **Repo:** <https://github.com/ollama/ollama>
- **Stars:** ~181k
- **Language:** Go
- **What it does:** Zero-config local model runner and server: pull, manage, and serve GGUF models (built on llama.cpp) with an OpenAI-compatible API, model library, and cross-platform desktop apps.
- **Why it's the best:** The best developer experience in local serving — one command to a running model on macOS, automatic Metal acceleration on M1, model lifecycle management, and broad tooling adoption. The standard "local backend" most harnesses integrate first.
- **Harness integration notes:** Treat it as the default local provider: your harness can auto-detect `http://localhost:11434` and enumerate pulled models. Its Modelfile/keep-alive semantics are a good reference for local model management UX.

LiteLLM

- **Repo:** <https://github.com/BerriAI/litellm>
- **Stars:** ~59k
- **Language:** Python (Rust core for gateway)
- **What it does:** Unified AI gateway and SDK that normalizes 100+ LLM APIs (OpenAI, Anthropic, Bedrock, Vertex, vLLM, Ollama, etc.) behind the OpenAI format, with load balancing, fallbacks, cost tracking, rate limiting, and guardrails.
- **Why it's the best:** The most complete model router/gateway in open source — a single integration point that lets a harness swap providers, route by cost/latency, and fail over between local (Ollama/vLLM) and hosted models without code changes.
- **Harness integration notes:** Use as the provider-abstraction layer, or steal its router strategies (latency-based routing, cooldowns, spend tracking) if building your own. Pairs naturally with a local M1 backend (Ollama/MLX) as a fallback route.

TabbyAPI (ExLlamaV2/V3)

- **Repo:** <https://github.com/theroyallab/tabbyAPI>
- **Stars:** ~1.4k
- **Language:** Python
- **What it does:** Lightweight OpenAI-compatible server for ExLlamaV2/V3, serving EXL2/GPTQ-quantized models with dynamic batching, prompt caching, LoRA support, and extensive sampler control.
- **Why it's the best:** The best single-GPU enthusiast engine — ExLlama's EXL2 quantization delivers excellent quality-per-bit and very fast decode on consumer NVIDIA cards, and TabbyAPI exposes it with a clean OAI API and low overhead.
- **Harness integration notes:** Worth supporting as a niche provider for NVIDIA consumer-GPU users; its sampler-override and prompt-caching design is instructive. Not applicable to Apple Silicon

(CUDA-only).

Honorable Mentions

- **ExLlamaV2** (<https://github.com/turboderp-org/exllamav2>) — the inference library behind TabbyAPI; EXL2 quantization format. CUDA-only.
- **MLC-LLM** (<https://github.com/mlc-ai/mlc-llm>) — Apache TVM-based universal compilation; deploys to Metal/Vulkan/WebGPU/mobile; good cross-platform story but less active than llama.cpp/MLX on Mac.
- **LocalAI** (<https://github.com/mudler/LocalAI>) — OpenAI-compatible multi-backend local gateway (llama.cpp, whisper, diffusers); good for multi-modal local serving.
- **RouteLLM** (<https://github.com/lm-sys/RouteLLM>) — LMSYS cost/quality routing framework with trained routers; ideas worth borrowing for model selection.
- **bitsandbytes** (<https://github.com/bitsandbytes-foundation/bitsandbytes>) — 8-bit/4-bit (NF4) quantization primitives used across the HF ecosystem.
- **AutoGPTQ** (<https://github.com/AutoGPTQ/AutoGPTQ>) / **AutoAWQ** (<https://github.com/casper-hansen/AutoAWQ>) — GPTQ/AWQ quantization toolchains (largely superseded by llm-compressor/ModelOpt but formats remain ubiquitous).
- **LMCache** (<https://github.com/LMCache/LMCache>) — KV-cache offloading/sharing layer for vLLM; cuts TTFT on long repeated contexts.
- **EAGLE** (<https://github.com/SafeAILab/EAGLE>) — reference speculative-decoding heads adopted by vLLM/SGLang.
- **LM Studio** — closed-source app (not a repo), but its MLX/llama.cpp dual-engine approach on Mac is the UX bar for local serving.

8 Agent Orchestration & Multi-Agent Frameworks

Orchestration is the heart of an LLM harness: how agents are modeled, how control flows between LLM calls and tools, how state persists across steps and crashes, and how work is fanned out to subagents in parallel. The projects below define the current state of the art across four sub-domains — graph/state-machine agent runtimes, role-based multi-agent collaboration, durable execution engines, and minimal code-first agent loops — and each offers concrete design ideas worth borrowing for a custom harness.

LangGraph

- **Repo:** <https://github.com/langchain-ai/langgraph>
- **Stars:** ~42k
- **Language:** Python (also JS/TS via langgraphjs)
- **What it does:** Low-level orchestration framework that models agents as stateful graphs: nodes are functions, edges define control flow, and a shared typed state object is threaded through execution with checkpointing.
- **Why it's the best:** The most influential mental model in the space — explicit state schema + reducer functions + super-step (Pregel-style) execution — gives deterministic, inspectable, resumable agent runs. First-class primitives for interrupts (human-in-the-loop), time-travel via checkpoints, durable execution, and subgraph composition. Trusted in production by Klarna, Replit, Elastic.
- **Harness integration notes:** Borrow the core ideas rather than the dependency: a typed state object with per-key reducers, a checkpointer interface for resume-after-crash, and `interrupt()` semantics for pausing mid-graph for human input. Its supervisor/swarm multi-agent patterns (in langgraph-supervisor) are the reference implementation for subagent delegation.

CrewAI

- **Repo:** <https://github.com/crewAIInc/crewAI>
- **Stars:** ~59k
- **Language:** Python
- **What it does:** Framework for orchestrating role-playing autonomous agents: you define Agents (role/goal/backstory), Tasks, and compose them into Crews (collaborative) or Flows (event-driven, deterministic control flow).
- **Why it's the best:** Highest-starred multi-agent framework; its Crews + Flows split is the cleanest articulation of the "autonomous collaboration vs. deterministic orchestration" duality. Standalone (no LangChain dependency since 2024), fast, and its YAML-config-driven agent definitions make agent teams declarative and shareable.
- **Harness integration notes:** Steal the role/goal/backstory prompt-scaffolding pattern for specializing subagents, the delegation tool design (agents asking each other for help), and the Flow abstraction (`@start`, `@listen`, `@router` decorators) for mixing deterministic control flow with autonomous crew execution.

OpenAI Agents SDK

- **Repo:** <https://github.com/openai/openai-agents-python>
- **Stars:** ~30k
- **Language:** Python (also JS/TS)
- **What it does:** Lightweight, provider-agnostic multi-agent framework built on a simple agent loop with four primitives: Agents (instructions + tools), Handoffs (agent-to-agent transfer), Guardrails (input/output validation), and Sessions (conversation state).
- **Why it's the best:** The most tasteful minimal API in the category — evolved from the experimental Swarm project, it proves that handoffs-as-tool-calls plus a runner loop covers most multi-agent patterns without graph machinery. Built-in tracing, guardrails that run in parallel with the agent, and 100+ model support via LiteLLM make it production-ready.
- **Harness integration notes:** The handoff primitive (an agent transfers control by emitting a special tool call, with optional input filters to trim history) is the single best pattern to copy for subagent delegation. Also study its session/pluggable-memory interface and streaming event model.

Pydantic AI

- **Repo:** <https://github.com/pydantic/pydantic-ai>
- **Stars:** ~20k
- **Language:** Python
- **What it does:** Agent framework from the Pydantic team built around type-safe, structured outputs: agents are generic over dependency injection and result types, with schema validation on every tool call and model response.
- **Why it's the best:** Brings FastAPI-grade ergonomics to agents — `Agent[DepsT, OutputT]` generics, dependency injection for tools, automatic retries on validation failure, and first-class support for every major model provider. Its graph library (pydantic-graph) adds typed state machines for complex flows, and it now ships durable execution integrations (Temporal, DBOS) plus AG-UI/A2A protocol support.
- **Harness integration notes:** Copy the dependency-injection design (tools receive a typed `RunContext[Deps]`) and the output-validator/retry loop — the cleanest way to force structured tool results. The way it layers durability as a pluggable wrapper around the agent loop (rather than baked in) is exactly how a harness should treat Temporal/Restate.

smolagents

- **Repo:** <https://github.com/huggingface/smolagents>
- **Stars:** ~29k
- **Language:** Python
- **What it does:** Hugging Face's deliberately tiny (~1k LOC core) agent library where agents think in code: the LLM writes Python actions that are executed directly, rather than emitting JSON tool calls.
- **Why it's the best:** The canonical reference for the CodeAct pattern — code actions compose better (loops, conditionals, variable reuse across tool calls) and benchmark ~30% fewer steps than JSON

tool-calling. Supports multi-agent hierarchies (managed agents), secure sandboxed execution (E2B, Docker), and any LLM via LiteLLM/transformers.

- **Harness integration notes:** Read it as the minimal viable agent loop: `run()` = while-loop of (call model → parse code → execute → append observation). Ideal for understanding what a harness's core loop needs and nothing more; its `ManagedAgent` pattern is a simple template for spawning subagents with isolated context.

Temporal

- **Repo:** <https://github.com/temporalio/temporal>
- **Stars:** ~23k
- **Language:** Go (server); SDKs in Python, TypeScript, Java, Go, .NET
- **What it does:** Durable execution platform: workflows are ordinary code whose progress is persisted via event sourcing, so they survive process crashes, deploys, and multi-day waits. Activities (side effects like LLM calls) get automatic retries, timeouts, and heartbeats.
- **Why it's the best:** The gold standard for durable execution — replay-based determinism, exactly-once activity semantics, and battle-tested at massive scale (Uber, Netflix, Snap). Its Python/TS SDKs are now explicitly marketed for AI agents: an agent loop becomes a workflow where each LLM/tool call is an activity, giving crash-proof hours-long agent runs for free.
- **Harness integration notes:** The key idea to internalize: separate deterministic orchestration (workflow code, replayed) from non-deterministic side effects (activities, recorded). Even without adopting Temporal, a harness checkpoint/replay design should follow this split. Its `temporalio.contrib.pydantic` AI-agent samples and the "durable agent" pattern are direct blueprints.

Restate

- **Repo:** <https://github.com/restatedev/restate>
- **Stars:** ~4.4k
- **Language:** Rust (server); SDKs in TypeScript, Python, Go, Java/Kotlin, Rust
- **What it does:** Durable execution as a lightweight single-binary service: handlers get durable promises, durable RPC/messaging, and virtual objects (stateful actors) — all journaled so code resumes exactly where it left off after any failure.
- **Why it's the best:** The rising star of durable execution — radically simpler ops story than Temporal (one Rust binary, no Cassandra/Postgres cluster), with a programming model (durable `ctx.run()`, awakeables, virtual objects keyed by ID) that maps beautifully onto agents: one virtual object per agent/conversation gives serialized, stateful, resumable agent instances. Restate explicitly documents AI-agent and durable-LLM-call patterns.
- **Harness integration notes:** Study the virtual-object model for per-session agent state (built-in inbox/serialization semantics solve concurrent-message ordering), and `ctx.run` for journaling LLM/tool calls. Its HTTP-based, infra-light deployment makes it a realistic embed target for a harness that wants durability without running a workflow cluster.

Inngest

- **Repo:** <https://github.com/inngest/inngest>
- **Stars:** ~5.8k
- **Language:** Go (server); SDKs in TypeScript, Python, Go
- **What it does:** Event-driven workflow orchestration platform: functions are composed of memoized steps (`step.run` , `step.sleep` , `step.waitForEvent`) that are retried and resumed individually, triggered by events or schedules.
- **Why it's the best:** The most developer-friendly step-function model — you write one function, wrap side effects in `step.*` , and get durability, retries, concurrency limits, and flow control without managing workers or queues. Strong TypeScript DX, self-hostable open-source server, and explicit support for agent workflows and AI SDK integration.
- **Harness integration notes:** The step-memoization API is the cleanest ergonomic design for durable agent loops: each LLM call/tool call becomes a named, memoized step, and `waitForEvent` gives elegant human-in-the-loop (pause until approval event arrives). Its event-driven trigger model is a good template for harness job scheduling.

Honorable Mentions

- **Microsoft Agent Framework** (<https://github.com/microsoft/agent-framework>, ~14k) — successor merging AutoGen + Semantic Kernel; graph workflows + A2A/MCP support, worth watching as AutoGen's official future.
- **AutoGen / AG2** (<https://github.com/microsoft/autogen> ~61k, <https://github.com/ag2ai/ag2> ~5k) — pioneered conversational multi-agent patterns (GroupChat, tool-using AssistantAgent); AutoGen v0.4's actor-model Core API is architecturally interesting, but the project has largely been superseded by Agent Framework.
- **Google ADK** (<https://github.com/google/adk-python>, ~22k) — code-first agent toolkit with strong multi-agent composition (sequential/parallel/loop agents) and A2A protocol; Gemini-optimized but model-agnostic.
- **Trigger.dev** (<https://github.com/triggerdotdev/trigger.dev>, ~16k) — TypeScript durable execution with excellent AI-agent ergonomics (streams, realtime, AI SDK integration).
- **Hatchet** (<https://github.com/hatchet-dev/hatchet>, ~8k) — Postgres-backed durable task queue with DAG workflows; good middle ground between Celery and Temporal.
- **DBOS Transact** (<https://github.com/dbos-inc/dbos-transact-py>, ~1.6k) — ultra-light durable workflows backed only by Postgres; clever "no orchestrator server" design.
- **Microsoft Semantic Kernel** (<https://github.com/microsoft/semantic-kernel>, ~29k) — enterprise-oriented agent/process framework, strong in C#/ .NET shops; its Process Framework is a notable workflow DSL.

9 Tool Use, MCP & Sandboxed Execution

The tool layer is where an LLM harness meets the real world: function calling, the Model Context Protocol (MCP) ecosystem that standardizes tool exposure, and the sandboxed execution environments that make agent-run code safe. A world-class harness needs a rich tool registry, first-class MCP client/server support, and a defensible isolation story for shell, filesystem, and code-interpreter tools.

modelcontextprotocol/servers

- **Repo:** <https://github.com/modelcontextprotocol/servers>
- **Stars:** ~90k
- **Language:** TypeScript / Python
- **What it does:** The official reference collection of MCP servers (filesystem, git, GitHub, Postgres, fetch, memory, sequential-thinking, etc.) plus the canonical index of community servers.
- **Why it's the best:** It is the de-facto standard catalog maintained by the MCP org itself; its reference servers define the idioms (tool schemas, resources, prompts, stdio vs. streamable-HTTP transports) every other server copies.
- **Harness integration notes:** Read the filesystem and git servers as templates for safe path-scoping and mutation-confirmation design; reuse the `.mcp.json` config format so users can plug any MCP server into your harness.

FastMCP

- **Repo:** <https://github.com/PrefectHQ/fastmcp>
- **Stars:** ~28k
- **Language:** Python
- **What it does:** The fast, Pythonic framework for building MCP servers and clients — decorators turn functions into tools; also offers proxying, composition, OpenAPI/FastAPI import, and auth.
- **Why it's the best:** Far more ergonomic than the raw SDK: automatic schema generation from type hints, server composition, middleware, and the ability to wrap arbitrary OpenAPI specs or existing MCP servers as proxies. Now the recommended high-level path on top of the official python-sdk.
- **Harness integration notes:** Use its client to talk to any MCP server from your harness, and its OpenAPI-to-MCP import to instantly convert REST APIs into tool surfaces without hand-writing wrappers.

awesome-mcp-servers

- **Repo:** <https://github.com/punkpeye/awesome-mcp-servers>
- **Stars:** ~95k
- **Language:** Markdown (curated list)

- **What it does:** The largest curated directory of MCP servers, clients, frameworks, and utilities, categorized by domain (databases, cloud, dev tools, browsers, finance...).
- **Why it's the best:** Most comprehensive and actively updated index of the MCP ecosystem; the fastest way to discover a production-ready server instead of building one.
- **Harness integration notes:** Mine it to seed your harness's tool marketplace/registry; its categorization is a good taxonomy for grouping tools by risk and capability.

MCP Context Forge (IBM)

- **Repo:** <https://github.com/IBM/mcp-context-forge>
- **Stars:** ~4.5k
- **Language:** Python
- **What it does:** An MCP gateway and proxy: federates many MCP servers and REST APIs behind one endpoint with auth, rate limiting, tool filtering, virtual-server composition, and an admin UI.
- **Why it's the best:** The most complete open-source MCP gateway — solves the "100 servers, one agent" problem with registry, transport bridging (stdio↔SSE↔streamable-HTTP), policy enforcement, and observability in one deployable.
- **Harness integration notes:** Put it between your harness and the tool fleet so the agent sees a single curated, permissioned tool endpoint; study its per-tool allow/deny and PII-filtering middleware for your own policy layer.

ToolHive

- **Repo:** <https://github.com/stacklok/toolhive>
- **Stars:** ~2.2k
- **Language:** Go
- **What it does:** A runtime/registry for deploying MCP servers securely: runs each server in an isolated container with least-privilege permissions, secrets injection, and a curated registry of vetted servers.
- **Why it's the best:** Security-first MCP operations — container isolation per server, permission profiles, and a CLI/UI that make running untrusted third-party tool servers practical.
- **Harness integration notes:** Adopt its threat model: never run MCP servers in-process; delegate to per-server sandboxes with explicit network/filesystem grants. Its registry metadata format is worth reusing.

E2B

- **Repo:** <https://github.com/e2b-dev/E2B>
- **Stars:** ~14k
- **Language:** TypeScript / Python / Go
- **What it does:** Open-source secure cloud sandboxes (Firecracker microVMs) purpose-built for AI agents: code interpreter, shell, filesystem, and long-running stateful environments with SDKs.

- **Why it's the best:** The leading agent-native sandbox: ~150ms cold starts, full VM isolation, snapshotting/pause-resume, and SDKs designed around LLM tool-call loops. Self-hostable infra included.
- **Harness integration notes:** The ideal backend for your harness's `run_code` / `shell` tool — give each agent session its own sandbox, stream stdout/stderr back as tool results, and snapshot state across turns.

Daytona

- **Repo:** <https://github.com/daytonaio/daytona>
- **Stars:** ~72k
- **Language:** Go / TypeScript
- **What it does:** Secure, elastic infrastructure for running AI-generated code — dev-environment manager turned agent-sandbox platform with SDKs for creating isolated workspaces on demand.
- **Why it's the best:** Battle-hardened dev-environment orchestration (OCI containers, pluggable providers) now packaged for agent workloads; strong on full-project workspaces rather than single snippets.
- **Harness integration notes:** Use when agents need whole-repo workspaces (install deps, run tests, edit many files) rather than snippet execution; its provider abstraction shows how to swap local Docker for remote isolation.

gVisor

- **Repo:** <https://github.com/google/gvisor>
- **Stars:** ~19k
- **Language:** Go
- **What it does:** An application kernel for containers: a userspace kernel that intercepts syscalls, giving OCI containers VM-grade isolation without VM overhead (runsc runtime, powers GKE Sandbox).
- **Why it's the best:** The most practical strong-isolation primitive you can embed locally — drops into Docker/containerd as a runtime, so any container-based tool sandbox gets a real security boundary for free.
- **Harness integration notes:** Run your harness's shell/code tools in Docker with `--runtime=runsc` for a meaningful security upgrade over plain containers; pair with Firecracker (below) when you need full microVMs.

Firecracker

- **Repo:** <https://github.com/firecracker-microvm/firecracker>
- **Stars:** ~37k
- **Language:** Rust
- **What it does:** AWS's microVM monitor: boots minimal KVM VMs in ~125ms with a tiny device model, built for multi-tenant serverless and container workloads (Lambda/Fargate run on it).

- **Why it's the best:** The gold standard for hardware-isolated, fast-start sandboxing; the foundation under E2B, Fly.io, and Kata-containers-style stacks.
- **Harness integration notes:** If you self-host agent code execution at scale, build on Firecracker (or consume it via E2B): one microVM per agent session gives kernel-level isolation with near-container density.

Honorable Mentions

- [modelcontextprotocol/python-sdk](#) — official low-level MCP SDK (~24k stars); use FastMCP unless you need protocol-level control.
- [tadata-org/fastapi_mcp](#) — zero-config FastAPI → MCP tool exposure (~12k stars).
- [StackBlitz WebContainers \(webcontainer-core\)](#) — Node.js-in-the-browser sandbox via WASM; proprietary API but the reference design for browser-side execution.
- [mcp-atlassian \(sooperset/mcp-atlassian\)](#) — exemplar of a deep, well-engineered domain MCP server (~8k stars).
- [Kata Containers](#) — VM-isolated OCI runtime; heavier alternative to gVisor for the same slot.

10 RAG, Embeddings & Retrieval

Retrieval quality is the single biggest lever on a harness's usefulness for repo/document knowledge: the best agent loop still fails if it retrieves the wrong context. This category covers the full pipeline a harness must embed — document parsing, chunking, embedding, hybrid retrieval with reranking, and graph-structured retrieval — and the projects below are the best-in-class, actively maintained building blocks for each stage.

RAGFlow

- **Repo:** <https://github.com/infiniflow/ragflow>
- **Stars:** ~91k
- **Language:** Python
- **What it does:** End-to-end open-source RAG engine with deep document understanding (DeepDoc), template-based chunking, grounded citations, and an agent layer on top of retrieval.
- **Why it's the best:** The most-starred RAG-specific project on GitHub and the reference implementation for production-grade ingestion: its DeepDoc parser handles complex PDF layouts, tables, and figures, and its chunking templates + hybrid retrieval with reranking are battle-tested. Extremely active (9k+ commits, frequent 2025-2026 releases).
- **Harness integration notes:** Study DeepDoc and its chunker/retriever pipeline as the gold standard for "documents in, grounded answers out"; usable as a service via REST API, or mine `deepdoc/` and `rag/` for parsing and fusion techniques to reimplement natively.

LlamaIndex

- **Repo:** https://github.com/run-llama/llama_index
- **Stars:** ~52k
- **Language:** Python
- **What it does:** The data framework for LLM applications — loaders, node parsers/chunkers, indexes (vector, keyword, knowledge graph), retrievers, query engines, and agentic RAG workflows.
- **Why it's the best:** The most comprehensive and modular retrieval toolkit: every pipeline stage (ingestion, chunking, retrieval, reranking, response synthesis) is a swappable component with hundreds of integrations. Its `SentenceSplitter`, `SemanticSplitterNodeParser`, `MarkdownNodeParser`, and recursive/auto-merging retrievers are the de-facto reference for chunking strategy design.
- **Harness integration notes:** Embed `llama-index-core` directly for indexing/retrieval abstractions, or copy its chunking and node-hierarchy designs (parent-child chunking, sentence-window retrieval) — these map perfectly onto repo/documentation knowledge bases.

LightRAG

- **Repo:** <https://github.com/HKUDS/LightRAG>

- **Stars:** ~40k
- **Language:** Python
- **What it does:** EMNLP 2025 graph-enhanced RAG: extracts entities/relations into a knowledge graph during indexing, then retrieves via dual-level (local + global) graph-vector hybrid queries.
- **Why it's the best:** The leading practical GraphRAG implementation — dramatically cheaper and faster to index than Microsoft GraphRAG while matching or beating it on multi-hop benchmarks. Pluggable storage (Neo4j, Postgres, NanoVectorDB) and a simple library API make it genuinely embeddable rather than a demo.
- **Harness integration notes:** Use as a library (`pip install lightrag-hku`) to add knowledge-graph retrieval over a codebase or doc corpus; its entity/relation extraction prompts and dual-level retrieval algorithm are worth studying even if you build your own graph layer.

Microsoft GraphRAG

- **Repo:** <https://github.com/microsoft/graphrag>
- **Stars:** ~36k
- **Language:** Python
- **What it does:** The original graph-based RAG system: builds an entity knowledge graph from a corpus, detects hierarchical communities (Leiden), pre-generates community summaries, and answers global "whole-corpus" questions via map-reduce over those summaries.
- **Why it's the best:** Defines the GraphRAG category and remains the reference for *global* sensemaking queries ("what are the main themes across this repo?") that pure vector search cannot answer. Its community-summarization indexing pipeline is the design everyone else (including LightRAG) reacts to.
- **Harness integration notes:** Adopt its local/global search split and community-summary indexing for corpus-level Q&A; expensive at index time, so a harness should run it as an offline indexing job and cache the graph + summaries.

Docling

- **Repo:** <https://github.com/docling-project/docling>
- **Stars:** ~66k
- **Language:** Python
- **What it does:** IBM's document-conversion toolkit: parses PDF, DOCX, PPTX, XLSX, HTML, images (OCR) into a unified `DoclingDocument` model with layout, table structure, and reading order; exports Markdown/JSON ready for chunking.
- **Why it's the best:** Winner of most 2025-2026 parser bake-offs for accuracy-per-cost on technical documents; local models (DocLayNet layout + TableFormer) mean no API dependency, and it ships a built-in `HybridChunker` that respects document structure. Backed by IBM and LF AI, very actively developed.
- **Harness integration notes:** The default ingestion front-end for a harness: `DocumentConverter` → `HybridChunker` → your vector store. Its structured document model is also a good internal representation to copy for code-aware parsing.

FlagEmbedding (BGE)

- **Repo:** <https://github.com/FlagOpen/FlagEmbedding>
- **Stars:** ~12k
- **Language:** Python
- **What it does:** BAAI's one-stop retrieval toolkit: the BGE family (bge-m3, bge-large/bge-base, bge-code) of embedding models, bge-reranker cross-encoders, and training/fine-tuning code for custom retrievers.
- **Why it's the best:** bge-m3 is the strongest open multilingual multi-functionality embedding (dense + sparse + multi-vector in one model, 8k context), and bge-reranker-v2-m3 is the standard open reranker — both consistently top MTEB/C-MTEB leaderboards among open weights. Also provides bge-code-v1, purpose-built for code retrieval.
- **Harness integration notes:** Run bge-m3 for embeddings and bge-reranker-v2-m3 as the rerank stage (via this lib, sentence-transformers, or a TEI/vLLM server); use bge-code-v1 for repo search. The repo's fine-tuning scripts let you specialize embeddings on your own corpus.

sentence-transformers

- **Repo:** <https://github.com/huggingface/sentence-transformers>
- **Stars:** ~19k
- **Language:** Python
- **What it does:** The canonical library for computing, training, and evaluating sentence/text embeddings and cross-encoder rerankers; hosts thousands of compatible models on the Hugging Face Hub.
- **Why it's the best:** The universal interface every embedding model (BGE, E5, GTE, Jina, Nomic) supports — one API for `encode()`, `CrossEncoder` reranking, sparse models, and Matryoshka/quantized embeddings. Now maintained under the Hugging Face org with active 2025-2026 releases (v5.x).
- **Harness integration notes:** Use it as the harness's embedding/rerank abstraction layer so the model is a config choice; its `CrossEncoder` API is the simplest way to add a rerank stage, and its evaluation module helps benchmark retrieval quality on your own data.

ColBERT (Stanford FutureData)

- **Repo:** <https://github.com/stanford-futuredata/ColBERT>
- **Stars:** ~3k
- **Language:** Python
- **What it does:** Late-interaction retrieval: encodes queries and documents into per-token vectors and scores with MaxSim, combining the precision of cross-encoders with the scalability of bi-encoders; includes ColBERTv2 indexing and PLAID engine.
- **Why it's the best:** The defining implementation of late interaction — the architecture behind Jina-ColBERT-v2, BGE-M3's multi-vector mode, and Vespa/Qdrant multi-vector support. Delivers state-of-the-art out-of-domain retrieval quality, and PLAID makes token-level indexes practical at scale.

- **Harness integration notes:** Use ColBERTv2 (or jina-colbert-v2 via the same paradigm) as the high-precision retrieval tier for code/doc search where embedding-only recall falls short; its token-level indexing ideas inform any custom multi-vector store design.

Honorable Mentions

- **Haystack** (<https://github.com/deepset-ai/haystack>, ~22k) — mature pipeline-oriented RAG framework; strong for production pipelines with explicit DAGs.
- **Marker** (<https://github.com/datalab-to/marker>, ~28k) — fast PDF→Markdown converter; excellent quality on scientific papers, good Docling alternative.
- **MinerU** (<https://github.com/openscalab/MinerU>, ~40k) — top-tier PDF extraction (layout, formulas, tables) from OpenDataLab; best-in-class for academic documents.
- **Unstructured** (<https://github.com/Unstructured-IO/unstructured>, ~12k) — broad-format ingestion library with partitioning/chunking; widely integrated into LangChain/LlamaIndex loaders.
- **Jina embeddings / jina-colbert-v2** (<https://github.com/jina-ai/ColBERT>) — strong multilingual embedding and late-interaction models with 8k context.
- **chonkie** (<https://github.com/chonkie-inc/chonkie>, ~3k) — lightweight, fast, dedicated chunking library (semantic, late, code chunkers) when you want chunking without a framework.
- **rank_bm25** (https://github.com/dorianbrown/rank_bm25, ~2k) — the standard minimal BM25 implementation for the lexical half of hybrid search + reciprocal rank fusion.

11 Prompt Engineering & System-Prompt Optimization

Prompts are the "source code" of an LLM harness: system prompts define agent behavior, structured-output constraints make tool calls reliable, and prompt attacks are the top security threat. This category covers the best open-source tooling for automatically optimizing prompts, enforcing structured outputs, defending against prompt injection, studying real-world system prompts, and evaluating prompt quality — the difference between a brittle demo harness and a production-grade one.

DSPy

- **Repo:** <https://github.com/stanfordnlp/dspy>
- **Stars:** ~38k
- **Language:** Python
- **What it does:** Framework for *programming* rather than prompting LMs — you write compositional Python modules (signatures, ChainOfThought, ReAct) and DSPy's optimizers (MIPROv2, BootstrapFewShot, GEPA) automatically compile them into optimized prompts, few-shot demos, or fine-tuned weights.
- **Why it's the best:** The definitive automatic prompt optimization framework, from Stanford NLP, backed by a strong research lineage (ICLR'24 paper; GEPA optimizer shows reflective prompt evolution can beat RL). Optimizers are model-agnostic and treat prompts as learnable hyperparameters of a program, not hand-tuned strings.
- **Harness integration notes:** Use DSPy as an offline "prompt compiler": define your agent's core calls as signatures, run an optimizer against an eval set, then export the optimized instructions/demos into your harness's system prompts. GEPA is especially relevant for harnesses that want self-improving agents.

TextGrad

- **Repo:** <https://github.com/zou-group/textgrad>
- **Stars:** ~3.7k
- **Language:** Python
- **What it does:** "Automatic differentiation via text" — an autograd-style engine where LLMs generate textual gradients (natural-language feedback) that backpropagate through a computation graph to optimize prompts, solutions, code, or molecules. Published in Nature (2025).
- **Why it's the best:** The cleanest implementation of the textual-gradient-descent paradigm, with a PyTorch-like API (`Variable` , `TextLoss` , `TGD` , `optimizer.step()`) that makes prompt optimization feel like training. Multimodal support and litellm-backed engines mean it works with any provider.
- **Harness integration notes:** Borrow its core loop — LLM-as-critic producing feedback that rewrites a prompt variable — to build a lightweight self-critique/self-refinement stage into your harness. Also a great reference for structuring "backward" feedback prompts.

Outlines

- **Repo:** <https://github.com/dottxt-ai/outlines>
- **Stars:** ~15.8k
- **Language:** Python
- **What it does:** Structured generation library that guarantees output format *during* decoding via constrained logits (JSON Schema, Pydantic models, regex, grammars, enums) rather than parsing/fixing after the fact. Works with Transformers, vLLM, Ollama, OpenAI, and more.
- **Why it's the best:** The reference implementation of constrained decoding — it compiles schemas into finite-state machines and masks invalid tokens, so malformed JSON is mathematically impossible. Trusted by NVIDIA, Cohere, HuggingFace, and integrated into vLLM's guided decoding.
- **Harness integration notes:** For local-model serving in your harness, Outlines (or vLLM's outlines/xgrammar backend) is the right way to enforce tool-call schemas. Study its logits-processor architecture if your harness runs its own inference loop.

Instructor

- **Repo:** <https://github.com/instructor-ai/instructor> (567-labs/instructor)
- **Stars:** ~13.9k
- **Language:** Python
- **What it does:** Pydantic-based structured-output client wrapper: define a `response_model`, and Instructor handles schema injection, validation, and automatic retries with error feedback across every major provider (OpenAI, Anthropic, Google, Ollama, Groq, ...).
- **Why it's the best:** The most ergonomic and widely adopted structured-extraction library for API-based models — one uniform interface over dozens of providers, with streaming of partial objects, validation retries, and battle-tested provider quirks handling.
- **Harness integration notes:** The fastest way to add reliable typed outputs to a harness that calls hosted APIs; its retry-with-validation-error pattern (feeding Pydantic errors back to the model) is a design worth copying even if you don't adopt the library.

promptfoo

- **Repo:** <https://github.com/promptfoo/promptfoo>
- **Stars:** ~25k
- **Language:** TypeScript
- **What it does:** Declarative testing and red-teaming for prompts, agents, and RAGs — YAML-configured eval matrices across models/providers, assertions (model-graded, semantic, programmatic), and an automated vulnerability scanner (jailbreaks, injection, PII leakage) with CI/CD integration.
- **Why it's the best:** The industry-standard prompt evaluation harness, used by OpenAI and Anthropic; uniquely combines prompt A/B evaluation with a full red-team suite, so regression testing and security testing live in one config.

- **Harness integration notes:** Adopt its eval-matrix model directly: version your harness's system prompts as promptfoo configs and gate releases on assertion pass rates. Its red-team plugins are also a checklist of attack classes your harness must defend against.

DeepEval

- **Repo:** <https://github.com/confident-ai/deepeval>
- **Stars:** ~18.3k
- **Language:** Python
- **What it does:** Pytest-style LLM evaluation framework with 40+ research-backed metrics (G-Eval, hallucination, answer relevancy, RAG faithfulness/contextual recall, toxicity, bias), synthetic dataset generation, and built-in benchmarks (MMLU, BigBenchHard, etc.).
- **Why it's the best:** The most complete Python-native eval framework — metrics are implemented from papers with LLM-judge pipelines, and it slots into existing pytest workflows, making prompt regressions testable like unit tests.
- **Harness integration notes:** Use it to score prompt variants inside your harness's CI, and mine its metric implementations (especially G-Eval's chain-of-thought judging and RAG faithfulness) as templates for building custom runtime quality monitors.

NeMo Guardrails

- **Repo:** <https://github.com/NVIDIA/NeMo-Guardrails>
- **Stars:** ~7.2k
- **Language:** Python
- **What it does:** NVIDIA's programmable guardrails toolkit: define input/output/dialog rails in the Colang modeling language to filter jailbreaks and prompt injections, enforce topical boundaries, fact-check outputs, and steer dialog flows around LLM calls.
- **Why it's the best:** The most mature open-source guardrail *runtime* — rails are declarative, composable, and sit as an interception layer around any LLM, with integrations for LangChain and major providers plus a jailbreak-detection model.
- **Harness integration notes:** Its architecture — a rail layer mediating every user→LLM and LLM→user message — is the pattern to copy for harness-level injection defense. Even without adopting Colang, its taxonomy of rail types (input/output/dialog/retrieval/execution) is a solid design checklist.

leaked-system-prompts

- **Repo:** <https://github.com/jujumilk3/leaked-system-prompts>
- **Stars:** ~14.9k
- **Language:** Markdown (collection)
- **What it does:** Curated, dated collection of leaked/extracted system prompts from major LLM products — Claude (incl. Claude Code, output styles), ChatGPT, Cursor, Windsurf, Devin, Manus, Gemini, Copilot, and dozens more, with verifiable sources.

- **Why it's the best:** The most comprehensive and actively maintained corpus of production system prompts — updated into 2026 with the latest agent products. Cited in academic papers; unmatched as ground truth for how frontier labs structure agent instructions, tool descriptions, and safety rules.
- **Harness integration notes:** Treat it as required reading, not a dependency: mine it for conventions (tool-use instructions, persona framing, refusal phrasing, context-management rules) when authoring your harness's system prompts, and diff versions over time to see how prompting practice evolves.

Honorable Mentions

- **Rebuff** (github.com/protectai/rebuff) — prompt-injection detection library (heuristics + ML model + canary tokens); development has slowed but the four-layer defense design is still a good reference.
- **LLM Guard** (github.com/protectai/llm-guard) — Protect AI's sanitization toolkit: scanners for prompts/outputs (injection, secrets, PII, toxicity) usable as a lightweight rail layer.
- **GEPA** (github.com/gepa-ai/gepa) — standalone reflective prompt-evolution optimizer from the DSPy lineage; useful if you want prompt evolution without the full DSPy framework.
- **PromptBreeder / APE implementations** — e.g. community reproductions of Google's PromptBreeder and the "Large Language Models Are Human-Level Prompt Engineers" (APE) algorithm; research-grade, worth studying for mutation-based prompt search but not production libraries.
- **Langfuse prompt management** (github.com/langfuse/langfuse) — open-source prompt CMS with versioning, labeling, and tracing integration if your harness needs prompt lifecycle management.

12 Evaluation, Observability & Tracing

<p>A harness without measurement is a black box: you cannot improve prompts, routing, or tool policies you cannot score, and you cannot debug agent trajectories you cannot replay. This category covers the two halves of measurement — evaluation frameworks (offline scoring of models, RAG pipelines, and agents against datasets and benchmarks) and observability platforms (tracing, cost/latency tracking, and online evaluation of live traffic). The best-in-class tools below converge on OpenTelemetry GenAI semantic conventions as the interchange format, so a harness that emits standard spans can plug into any of them.</p>

Langfuse

- **Repo:** <https://github.com/langfuse/langfuse>
- **Stars:** ~34.7k
- **Language:** TypeScript (with Python/JS SDKs)
- **What it does:** Open-source LLM engineering platform combining tracing, prompt management, datasets, evaluations, and cost/latency dashboards in one self-hostable stack.
- **Why it's the best:** The most complete open platform — nested trace/span/observation model designed for agents, native OTEL GenAI endpoint, prompt registry with versioning and A/B labels, scoring (user, LLM-as-judge, programmatic), and dataset-driven eval runs. Very actively developed (~9.4k commits, weekly releases through 2025-2026).
- **Harness integration notes:** Emit traces via its OTEL-compatible ingestion endpoint or the Python/TS SDK decorators; reuse its data model (trace → observation → score → dataset run) as the schema for your own harness telemetry, even if you self-host nothing else.

Arize Phoenix

- **Repo:** <https://github.com/Arize-ai/phoenix>
- **Stars:** ~11.5k
- **Language:** Python (TypeScript UI)
- **What it does:** AI observability and evaluation platform built natively on OpenTelemetry: tracing, LLM-as-judge evals, RAG analysis, embedding drift visualization, and experiments.
- **Why it's the best:** Purest OTEL-native design — it is both a collector and UI for GenAI spans, so any harness already emitting OpenInference/OTEL GenAI spans gets full tracing for free. Runs as a single `pip install` local process, ideal for dev-loop debugging, and ships strong built-in eval templates (hallucination, relevance, toxicity).
- **Harness integration notes:** Point your OTEL exporter at Phoenix's OTLP endpoint during development; study `openinference-instrumentation-*` packages for how to auto-instrument LLM/agent frameworks without invasive code changes.

Opik (Comet)

- **Repo:** <https://github.com/comet-ml/opik>
- **Stars:** ~22.1k
- **Language:** Python/Java backend, TypeScript UI
- **What it does:** End-to-end LLM evaluation and observability platform: tracing, annotated datasets, LLM-as-judge metrics, experiments, and production monitoring dashboards.
- **Why it's the best:** Strongest blend of eval rigor and tracing among the newer platforms — built-in metric library (hallucination, G-Eval, moderation), pytest-style experiment runner for CI, and agent-optimization tooling (Opik Optimizer). Fast-growing and very actively maintained in 2025-2026.
- **Harness integration notes:** Use its `@track` decorator / SDK to capture agent spans and its `evaluate()` API to run dataset experiments from your harness; its metric implementations are a good reference for judge-prompt design.

DeepEval

- **Repo:** <https://github.com/confident-ai/deepeval>
- **Stars:** ~18.3k
- **Language:** Python
- **What it does:** Pytest-style LLM evaluation framework with 40+ metrics (G-Eval, hallucination, answer relevancy, RAG metrics, agent/tool-use metrics, safety) runnable locally or against the Confident AI cloud.
- **Why it's the best:** The most ergonomic "unit test your LLM app" experience — `assert_test()` semantics drop directly into existing pytest suites, making evals part of CI rather than a separate workflow. Broad metric coverage including agent-specific ones (tool correctness, task completion).
- **Harness integration notes:** Embed its metrics as scoring functions inside your harness's eval runner; adopt its pattern of treating eval cases as test functions with thresholds so regressions fail CI.

Ragas

- **Repo:** <https://github.com/explodinggradients/ragas>
- **Stars:** ~15.8k
- **Language:** Python
- **What it does:** Reference-free evaluation library focused on RAG (faithfulness, context precision/recall, answer correctness) plus agent and tool-use metrics, with synthetic test-set generation.
- **Why it's the best:** The de-facto standard for RAG evaluation metrics — its metric definitions are the ones cited in papers and copied by other frameworks. Synthetic dataset generation from your own documents solves the cold-start problem of eval data.
- **Harness integration notes:** Import its metric functions to score retrieval quality in a RAG-enabled harness; use its TestsetGenerator to bootstrap eval datasets from your harness's document corpus.

promptfoo

- **Repo:** <https://github.com/promptfoo/promptfoo>
- **Stars:** ~25.2k
- **Language:** TypeScript
- **What it does:** CLI-first eval and red-teaming tool: declarative YAML configs define prompts, providers, and assertions; runs matrix comparisons across models and scans for vulnerabilities (jailbreaks, prompt injection, PII leakage).
- **Why it's the best:** Best fit for harness builders because it's config-driven and CI-native — a single `promptfoo eval` gives side-by-side model/prompt matrices with pass/fail assertions, and its red-team module is the most comprehensive open-source LLM vulnerability scanner.
- **Harness integration notes:** Drive your harness as a custom provider (script or HTTP) so every prompt/model/config change is benchmarked identically; wire `promptfoo eval` into CI as a regression gate.

LM Evaluation Harness (EleutherAI)

- **Repo:** <https://github.com/EleutherAI/lm-evaluation-harness>
- **Stars:** ~14k
- **Language:** Python
- **What it does:** The standard academic benchmarking framework: 60+ standardized tasks (MMLU, HellaSwag, GSM8K, etc.) with reproducible few-shot prompting, supporting HF models, vLLM, and API endpoints.
- **Why it's the best:** It is the backbone of the Hugging Face Open LLM Leaderboard and the reference implementation for reproducible base-model benchmarking — results from it are comparable across the entire field.
- **Harness integration notes:** Use it to baseline whatever model your harness routes to; its task/registry architecture (YAML-defined tasks with metrics) is a proven pattern for a pluggable eval-task system.

OpenLLMetry (Traceloop)

- **Repo:** <https://github.com/traceloop/openllmetry>
- **Stars:** ~7.4k
- **Language:** Python (with JS sibling repo)
- **What it does:** OpenTelemetry instrumentation packages for LLM providers, vector DBs, and frameworks (OpenAI, Anthropic, LangChain, LlamaIndex, Pinecone...), emitting standardized GenAI spans to any OTLP backend.
- **Why it's the best:** The reference implementation of the OTel GenAI semantic conventions — vendor-neutral, composes with existing observability stacks (Jaeger, Grafana, Datadog), and heavily contributed to the official `opentelemetry-instrumentation` GenAI effort.

- **Harness integration notes:** Rather than writing your own span attributes, adopt these instrumentations (or mirror their attribute schema) so your harness traces interoperate with every OTel-compatible backend.

Honorable Mentions

- **OpenLIT** (<https://github.com/openlit/openlit>, ~2.8k) — OpenTelemetry-native LLM/GPU observability with built-in cost tracking; lighter-weight alternative to Langfuse.
- **Helicone** (<https://github.com/Helicone/helicone>, ~6.2k) — proxy-based LLM observability with cost/latency tracking, caching, and rate limits; one-line integration via base-URL swap.
- **SWE-bench** (<https://github.com/princeton-nlp/SWE-bench>, ~5.9k) — the canonical agentic coding benchmark (real GitHub issues); essential for scoring a coding harness end-to-end.
- **OSWorld** (<https://github.com/xlang-ai/OSWorld>, ~3.1k) — benchmark for multimodal/computer-use agents operating real OS environments.
- **HELM** (<https://github.com/stanford-crfm/helm>, ~2.9k) — Stanford's holistic, multi-scenario model evaluation framework; rigorous but heavier to run.
- **OpenAI Evals** (<https://github.com/openai/evals>, ~19.5k) — historically important eval registry and framework; largely superseded but its eval-authoring patterns are still instructive.
- **OpenTelemetry Semantic Conventions** (<https://github.com/open-telemetry/semantic-conventions>) — the GenAI span/attribute standard itself; read this before designing your trace schema.
- **TruLens** (<https://github.com/truera/trulens>) — feedback-function-based eval and tracking, especially for RAG triad metrics.

13 Planning, Reasoning & Self-Improvement

A world-class LLM harness lives or dies by its control loop: how it decomposes goals into plans, searches over candidate reasoning paths, critiques its own output, and accumulates reusable skills over time. The projects below are the canonical reference implementations for the major planning paradigms (ReAct, Plan-and-Execute, Tree-of-Thoughts, LATs/MCTS), self-improvement loops (Reflexion, Self-Refine), long-horizon coding agents (SWE-agent), and continual-learning skill libraries (Voyager). Each is a real, verified GitHub repo whose patterns can be lifted directly into a harness.

LangGraph

- **Repo:** <https://github.com/langchain-ai/langgraph>
- **Stars:** ~41.8k
- **Language:** Python (with a JS/TS twin, langgraphjs)
- **What it does:** Graph-based agent orchestration framework where agent control flow is modeled as a state machine of nodes and edges, with first-class support for cycles, branching, persistence (checkpoints), human-in-the-loop interrupts, and time-travel debugging.
- **Why it's the best:** It is the most widely adopted production-grade implementation of cyclic agent graphs — the natural home for ReAct loops, plan-and-execute, reflection loops, and tree-search control flow. The explicit state-graph model (vs. hidden loop logic) is exactly the abstraction a harness needs for inspectable, resumable long-horizon runs.
- **Harness integration notes:** Adopt its core ideas: a typed shared state object, nodes as pure-ish functions over state, conditional edges for routing, and checkpointing for crash recovery. Its prebuilt `create_react_agent` and plan-and-execute tutorials are direct blueprints.

SWE-agent

- **Repo:** <https://github.com/SWE-agent/SWE-agent>
- **Stars:** ~20.3k
- **Language:** Python
- **What it does:** Agent that takes a GitHub issue and autonomously fixes it in a sandboxed repo using a purpose-built Agent-Computer Interface (ACI): custom shell commands, a file viewer/editor with lint-gated edits, and search tools. NeurIPS 2024; top results on SWE-bench.
- **Why it's the best:** The definitive reference for long-horizon, TODO-driven coding agents in the Claude Code style. Its key insight — that tool/interface design (compact observation, guardrails on edits, one-action-per-turn) matters more than prompt cleverness — is the single most transferable lesson for harness builders.
- **Harness integration notes:** Study `sweagent/tools` and its trajectory/config system: how it truncates observations, enforces edit syntax, blocks multi-edit commands, and manages context window over 100+ step trajectories. Directly reusable as a sub-agent for code tasks.

Voyager

- **Repo:** <https://github.com/MineDojo/Voyager>
- **Stars:** ~7.2k
- **Language:** Python (+ Node.js for the Minecraft bridge)
- **What it does:** Open-ended lifelong-learning agent (Minecraft) with three pillars: an automatic curriculum proposing progressively harder tasks, an ever-growing skill library of executable code (indexed by embedding for retrieval), and an iterative prompting loop that folds in environment feedback, execution errors, and self-verification.
- **Why it's the best:** The canonical implementation of continual learning via a code-as-skill library — skills are compositional, interpretable, and transferable to new worlds, which compounds capability without fine-tuning. No other open project demonstrates the full write-verify-store-retrieve-reuse skill loop as cleanly.
- **Harness integration notes:** The `voyager/agents` split (curriculum, action, critic, skill manager) is a ready-made module decomposition. The skill-library pattern — store verified code snippets with descriptions, retrieve by embedding similarity, compose at plan time — ports directly to any tool-using harness.

Tree of Thoughts (official)

- **Repo:** <https://github.com/princeton-nlp/tree-of-thought-llm>
- **Stars:** ~6.1k
- **Language:** Python
- **What it does:** Official NeurIPS 2023 implementation of Tree of Thoughts: deliberate problem solving by generating multiple intermediate "thoughts," evaluating them with the LM (value/vote prompts), and searching the tree with BFS/DFS. Includes Game of 24, creative writing, and mini-crosswords tasks.
- **Why it's the best:** The reference implementation for search-based reasoning over LM outputs. Its clean separation — `Task` interface (`get_status`, `get_outputs`) vs. search method (BFS with state scoring) — is the minimal correct skeleton for any tree-search reasoning feature.
- **Harness integration notes:** Implement its two abstractions: a task adapter exposing `propose -> evaluate -> search`, and pluggable value prompts. Note the practical lesson: ToT costs many more calls than CoT, so a harness should gate tree search behind difficulty heuristics.

ReAct (official)

- **Repo:** <https://github.com/ysymyth/ReAct>
- **Stars:** ~4.2k
- **Language:** Python
- **What it does:** Official code for "ReAct: Synergizing Reasoning and Acting in Language Models" (ICLR 2023): interleaved Thought/Action/Observation traces on HotpotQA (Wikipedia API), FEVER, ALFWorld, and WebShop.

- **Why it's the best:** The foundational thought-action-observation loop that nearly every modern agent harness implements. The repo shows the exact prompt formats and few-shot trace structure that make interleaved reasoning/acting work, including PaLM-scale and fine-tuning variants.
- **Harness integration notes:** Copy the trace schema (Thought/Action/Observation triples) as your harness's core step record — it doubles as a debugging log and training data. The Wikipedia-env wrapper shows how to keep tool APIs trivially small (search/lookup/finish).

Reflexion

- **Repo:** <https://github.com/noahshinn/reflexion>
- **Stars:** ~3.3k
- **Language:** Python
- **What it does:** Official NeurIPS 2023 implementation of Reflexion: verbal reinforcement learning where an agent, after a failed episode, generates a natural-language self-reflection stored in episodic memory and prepended to the next attempt's context. Covers decision-making (AlfWorld), reasoning (HotpotQA), and programming (HumanEval) tasks.
- **Why it's the best:** The definitive self-critique loop: evaluator -> self-reflection model -> episodic memory buffer. Its `ReflexionStrategy` enum (NONE / LAST_ATTEMPT / REFLEXION / LAST_ATTEMPT_AND_REFLEXION) is a clean taxonomy of how much failure context to feed back — directly measurable trade-offs a harness should expose.
- **Harness integration notes:** Implement reflection as a post-episode hook: on failure signal (test failure, task timeout), generate a bounded self-critique, store it in a sliding memory, and inject into the next attempt's system context. The `programming_runs` code shows the unit-test-as-reward-signal pattern.

LATS (Language Agent Tree Search)

- **Repo:** <https://github.com/andyz245/LanguageAgentTreeSearch>
- **Stars:** ~0.9k
- **Language:** Python
- **What it does:** Official implementation of "Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models" (ICML 2024): Monte Carlo Tree Search over LM agents, combining ReAct-style acting, Reflexion-style reflection, and ToT-style search with environment feedback and value functions. Evaluated on HotpotQA, WebShop, ALFWorld, and programming.
- **Why it's the best:** The most complete open-source MCTS-for-LLMs implementation — selection, expansion, simulation, and backpropagation where nodes are full agent states (thought-action-observation histories). It empirically subsumes ReAct, Reflexion, and ToT, making it the right reference when a harness needs best-of-N search with external feedback.
- **Harness integration notes:** Use it to understand the MCTS node representation for agents (state = trajectory prefix, value = LM self-eval + env reward) and the practical knobs: branching factor, depth, and reflection-on-backpropagation. Expensive per query — reserve for high-stakes steps.

Self-Refine

- **Repo:** <https://github.com/madaan/self-refine>
- **Stars:** ~0.8k
- **Language:** Python
- **What it does:** Official NeurIPS 2023 implementation of Self-Refine: iterative refinement where a single LM generates output, critiques it with a feedback prompt, then refines it — repeating until the feedback is neutral or a budget is hit. No training, no external reward; tasks span dialogue, code optimization, sentiment reversal, math, and more.
- **Why it's the best:** The simplest possible self-improvement loop (generate -> feedback -> refine, same model throughout) and therefore the easiest to embed as a harness primitive. Its per-task prompt triples (generation/feedback/refine) are a proven template for critique-driven iteration without any environment signal.
- **Harness integration notes:** Implement as an optional `refine(output, criteria, max_iters)` wrapper around any generation step. Key lessons: feedback prompts must be actionable and specific, and you need a stopping condition (score threshold or iteration cap) to avoid refinement loops that degrade output.

Honorable Mentions

- **kyegomez/tree-of-thoughts** (~1.5k) — plug-and-play ToT/ToT-v2 with more search strategies; less rigorous than the official repo but more batteries-included.
- **OpenBMB/ChatDev** (~34k) — multi-agent software company simulation; a reference for role-based plan decomposition across specialized agents.
- **gpt-engineer-org/gpt-engineer** (~55k) — clarify-then-generate planning flow for whole-codebase generation; good reference for spec-first planning UX.
- **langchain-ai/langchain** `plan-and-execute` **experimental agent** — the original BabyAGI-inspired planner/executor split; largely superseded by LangGraph but historically important.
- **microsoft/autogen (AG2)** — multi-agent conversation framework whose `AssistantAgent / UserProxyAgent` loop encodes plan-critique cycles between agents.
- **significant-gravitas/AutoGPT** — the original TODO-list-driven autonomous agent; largely historical now, but its continuous-mode task queue pattern influenced every long-horizon harness since.